





UNIVERSITÉ DE MONTPELLIER

Faculté des Sciences — Master Informatique

Mémoire de fin d'études

Master 2 Informatique — Parcours Génie Logiciel (GL)

Réduction de la dette technique dans un environnement de production

Mémoire rédigé à l'issue de deux années d'alternance

Travaux réalisés dans le cadre d'un contrat d'apprentissage au sein de
Free Mobile et Beemo Technologie.





Présenté par

Jérémy Hurel

Encadrement

David Desmarest — *tuteur entreprise*
Mickaël Montassier — *tuteur universitaire*

Année universitaire 2024 – 2026

Remerciements

Je tiens en premier lieu à remercier David Desmarest pour m'avoir donné l'opportunité de réaliser mon alternance, d'abord chez Free Mobile, puis pour m'avoir proposé un poste chez Beemo Technologie afin de la mener à son terme. Je le remercie pour sa pédagogie tout au long de ces deux années et pour tout ce qu'il m'a appris dans le domaine du génie logiciel, en particulier au travers des missions qu'il m'a confiées et qui m'ont permis de progresser.

Je souhaite également remercier Mickaël Montassier, qui a su se rendre joignable en toutes circonstances pour répondre à mes questions, notamment lorsque s'est posée celle de quitter ou non Free Mobile.

Enfin, je remercie l'Université de Montpellier d'avoir mis en place son *job dating*, qui m'a permis d'obtenir mon poste chez Free Mobile, ainsi que l'ensemble des collègues que j'ai côtoyés aussi bien chez Free que chez Beemo. Tous se sont montrés ouverts et disponibles pour répondre à mes questions, sans jamais faire de différence liée à mon statut d'alternant.

Résumé

Ce mémoire retrace une alternance de deux années menée au sein de deux entreprises : Free Mobile, opérateur de télécommunications du groupe Iliad, puis Beemo Technologie, éditeur d'une solution souveraine de sauvegarde de données. Un même fil conducteur relie ces deux expériences, à savoir la maîtrise de la dette technique et les enjeux business qui lui sont associés.

Chez Free Mobile, au sein de l'équipe en charge du logiciel de valorisation des appels, les missions ont consisté à outiller la reprise de contrôle d'un logiciel critique : automatisation de la duplication de la supervision Grafana, conception d'un framework de test d'intégration et de non-régression à scénarios déclaratifs, développement d'une extension VS Code (serveur de langage et débogueur) au service de ce framework, portage de celui-ci vers les *forks* legacy du moteur, et maîtrise du coût de compilation d'une base C++ conséquente. Chez Beemo Technologie, le travail s'est poursuivi sur deux fronts : une démarche de rétro-ingénierie pour rendre reproductible la compilation d'un agent avec le SDK d'un constructeur, puis le développement d'une fonctionnalité de production, l'immuabilité des jeux de sauvegarde, injectée dans un firmware fortement endetté sans aggraver son état.

À travers ces missions, le mémoire montre que la dette technique n'est pas qu'un problème de code : c'est un arbitrage économique permanent entre *time-to-market* et soutenabilité du logiciel, dont le traitement engage autant l'outillage et l'architecture que la motivation des équipes.

Mots-clés : dette technique, tests automatisés, non-régression, intégration continue, rétro-ingénierie, qualité logicielle.

Table des matières

Remerciements **i**

Résumé **ii**

1 Introduction **1**
1.1 Contexte de l'alternance 1
1.2 Le fil conducteur : la dette technique 1
1.3 Problématique 2
1.4 Plan du mémoire 2

2 Présentation des entreprises **3**
2.1 Free Mobile 3
2.2 Beemo Technologie 4
2.3 Positionnement des missions 4

3 Environnement technique **6**
3.1 Un socle C++ entouré de langages satellites 6
3.2 Compiler, versionner, intégrer 7
3.3 Stockage et supervision 7
3.4 Organisation du travail 8

4 Appropriation scientifique : la dette technique et ses enjeux business **9**
4.1 Définition et métaphore 9
4.2 Typologie de la dette technique 10
4.3 Enjeux business 10
4.4 Retour d'expérience : stratégies d'adressage et dimension sociale 11
4.4.1 Une même logique d'adressage incrémental 11
4.4.2 Des dettes de nature différente 11
4.4.3 L'enjeu social du traitement de la dette 12
4.5 Leviers de réduction 12
4.6 Grille de lecture des missions 13

5 Outiller la productivité et standardiser l'environnement de développement 15
5.1 Un outil de duplication de configuration Grafana 16
5.1.1 Contexte et dette technique initiale 16
5.1.2 Objectif de la mission 16
5.1.3 Conception et choix techniques 16
5.1.4 Réalisation 17
5.1.5 Difficultés rencontrées 17
5.1.6 Résultats et bénéfice business 17
5.2 Maîtrise du coût de compilation du Valo Engine 18
5.2.1 Contexte et dette technique initiale 18
5.2.2 Objectif de la mission 18
5.2.3 Conception et choix techniques 19
5.2.4 Réalisation 19
5.2.5 Difficultés rencontrées 19
5.2.6 Résultats et bénéfice business 20
5.3 Rétro-ingénierie pour la compilation avec un SDK 20
5.3.1 Contexte et dette technique initiale 20
5.3.2 Objectif de la mission 21
5.3.3 Démarche de rétro-ingénierie 21
5.3.4 Réalisation 21
5.3.5 Difficultés rencontrées 22
5.3.6 Résultats et bénéfice business 22

6 Rendre testable un logiciel legacy : résorber les régressions et s'en prémunir 23
6.1 Un framework de test d'intégration et de non-régression 24
6.1.1 Contexte et dette technique initiale 24
6.1.2 Objectif de la mission 24
6.1.3 Conception et choix techniques 24
6.1.4 Réalisation 26
6.1.5 Difficultés rencontrées 29
6.1.6 Résultats et bénéfice business 30
6.2 Outillage du framework : une extension VS Code (LSP et débogueur) . . . 30
6.2.1 Contexte et dette technique initiale 30
6.2.2 Objectif de la mission 31
6.2.3 Conception et choix techniques 31
6.2.4 Réalisation 32
6.2.5 Difficultés rencontrées 32
6.2.6 Résultats et bénéfice business 33
6.3 Portage du framework vers les forks legacy du Valo Engine 33

6.3.1 Contexte et dette technique initiale 33
6.3.2 Objectif de la mission 34
6.3.3 Conception : factorisation et paramétrage 34
6.3.4 Réalisation 35
6.3.5 Difficultés rencontrées 35
6.3.6 Expérimentation : l'IA agentique au service de la transmission . . . 35
6.3.7 Résultats et bénéfice business 36

7 Développer du code à visée de production dans un existant endetté 38
7.1 Développement d'une fonctionnalité en code de production 39
7.1.1 Contexte et besoin 39
7.1.2 Objectif de la mission 39
7.1.3 Conception et intégration 40
7.1.4 Réalisation 40
7.1.5 Difficultés rencontrées 42
7.1.6 Résultats et bénéfice business 42

8 Conclusion 43
8.1 Bilan fonctionnel 43
8.2 Retour sur le fil rouge : la dette technique 44
8.3 Apports de l'alternance 44
8.4 Perspectives 45

Glossaire des acronymes 46

Glossaire 47

A Annexes 50
A.1 Chronologie des missions 50

Chapitre 1

Introduction

1.1 Contexte de l'alternance

Ce mémoire rend compte de l'alternance effectuée dans le cadre de mon Master Informatique à la Faculté des Sciences de l'Université de Montpellier. Réalisée sur deux années, de juillet 2024 à juin 2026, elle présente une particularité : elle s'est déroulée dans **deux entreprises successives**. J'ai d'abord intégré **Free Mobile**, opérateur de télécommunications du groupe Iliad, au sein de l'équipe en charge du logiciel de valorisation des appels ; j'ai ensuite rejoint **Beemo Technologie**, éditeur d'une solution de sauvegarde de données, pour y achever mon alternance en tant que développeur. La soutenance intervenant avant le terme du contrat, certains travaux présentés ici sont encore en cours au moment de la rédaction.

1.2 Le fil conducteur : la dette technique

Les missions techniques présentées dans ce mémoire semblent hétérogènes : outil de duplication de supervision, maîtrise d'une chaîne de compilation, rétro-ingénierie d'un Software Development Kit (SDK), framework de test et extension d'éditeur à son service, portage de ce framework vers des forks legacy, développement d'une fonctionnalité de production. Un même enjeu traverse pourtant l'ensemble de cette alternance : la dette technique. Dans les deux entreprises, j'ai été confronté à des logiciels critiques pour l'activité, mais dégradés par les années : sources partiellement non versionnées, absence de tests, *forks* multipliés, chaînes de compilation opaques, savoir concentré sur des personnes ayant quitté l'entreprise ou sur le point de le faire.

Or cette dette n'est jamais un problème purement technique. Elle se paie en ralentissement des livraisons, en risque de régression sur des composants dont la défaillance se chiffre en argent réel, et en démotivation des équipes. La traiter relève donc d'un arbitrage

économique : continuer à livrer de la valeur pour rester compétitif, tout en investissant dans la soutenabilité du logiciel.

1.3 Problématique

La question centrale que ce mémoire cherche à éclairer peut se formuler ainsi : *comment des actions d'ingénierie ciblées (outillage, tests, maîtrise de la chaîne de production, architecture d'isolation) permettent-elles de maîtriser la dette technique d'un logiciel en exploitation, et en quoi cette maîtrise se traduit-elle en valeur pour l'entreprise ?* Chaque mission présentée dans la suite constitue une réponse partielle à cette question, sur un terrain et avec des contraintes à chaque fois différents.

1.4 Plan du mémoire

La suite du mémoire s'ouvre sur la présentation des deux entreprises et du positionnement de mes missions dans chacune d'elles (chapitre 2), prolongée par une description succincte de l'environnement technique commun à ces travaux (chapitre 3). Le chapitre 4 construit ensuite la grille de lecture de l'ensemble : il définit la dette technique, en distingue les principales formes et montre que son traitement relève d'un arbitrage économique autant que technique.

Les travaux effectués occupent les trois chapitres suivants. Plutôt que de suivre la chronologie, restituée par le diagramme de Gantt en annexe A, ils sont regroupés selon la nature du levier actionné contre la dette, chaque mission formant une section. Le chapitre 5 montre d'abord comment l'**outillage** du quotidien des développeurs rembourse une dette faite de procédures manuelles et de savoir-faire non capitalisés, de la duplication de la supervision Grafana à la rétro-ingénierie d'un SDK constructeur, en passant par la maîtrise du coût de compilation d'un moteur C++. Le chapitre 6 suit ensuite la construction d'un filet de non-régression autour d'un logiciel critique devenu **difficile à tester** : un framework de test à scénarios déclaratifs y voit le jour, une extension VS Code en ouvre l'usage à toute l'équipe, avant que l'ensemble ne soit porté vers les forks legacy du moteur. Le chapitre 7 déplace enfin la question sur le terrain du **code de production** : livrer une fonctionnalité nouvelle au cœur d'un firmware endetté, sans aggraver son état. La conclusion dresse le bilan fonctionnel de ces missions et revient sur les apports de ces deux années d'alternance.

Chapitre 2

Présentation des entreprises

2.1 Free Mobile

Free Mobile est un opérateur de télécommunications du groupe Iliad, fondé par Xavier Niel. L'entreprise est notamment connue pour être entrée sur le marché de la téléphonie mobile de manière agressive, en particulier avec son forfait à deux euros. Au-delà de ses offres commerciales, Free Mobile fait partie des rares opérateurs à maîtriser l'ensemble de leurs logiciels en interne : tout leur cœur de réseau y est développé, de même que la valorisation, et l'entreprise dispose en outre d'une section R&D dédiée à ses propres équipements matériels.

J'ai intégré l'équipe Valo, rattachée au Core SI. Cette équipe a la charge du logiciel de valorisation, qui calcule la valeur de l'ensemble des appels passés en téléphonie mobile sur les différents réseaux.

À mon arrivée, ce logiciel se trouvait dans un contexte de dette technique élevée : l'ingénieur qui en assurait la maintenance avait quitté l'entreprise, et plusieurs règles de génie logiciel n'avaient pas été respectées : une partie des sources, par exemple, n'était pas versionnée sous Git. L'équipe, montée et dirigée par David Desmarest, s'est vu confier une mission de reprise de contrôle de ce logiciel.

Mon rôle y était transverse au travail de l'équipe de développement : supporter celle-ci en produisant des outils internes, au premier rang desquels le framework de test détaillé à la section 6.1.

2.2 Beemo Technologie

Beemo Technologie est une entreprise d'une vingtaine d'années spécialisée dans la sauvegarde de données. Sa particularité est de ne pas se limiter à un logiciel installé sur une machine du client, mais de proposer une solution clé en main : des boîtiers (*box*) déployés au sein de l'entreprise, voire chez son dirigeant, qui assurent la sauvegarde et la duplication des données, et donc leur redondance en cas de perte.

L'entreprise compte une trentaine d'employés et dispose de clients et de partenaires partout dans le monde ; elle se développe actuellement sur le continent africain. Une partie des effectifs est dédiée à une *hotline*, reconnue pour sa réactivité et son efficacité dans la résolution des problèmes. Beemo se distingue enfin par le caractère souverain de sa solution de sauvegarde, un atout particulièrement recherché dans le contexte actuel.

Mon arrivée chez Beemo a fait suite à une proposition de David Desmarest, qui, après avoir quitté Free Mobile, y avait pris le *lead* de l'équipe de développement. Ma décision de le suivre a répondu à plusieurs facteurs. L'équipe que je quittais traversait une restructuration qui faisait peser une incertitude sur son devenir, sur laquelle le management communiquait peu ; je m'y retrouvais paradoxalement parmi les développeurs les plus anciens, la plupart des autres ayant rejoint l'équipe après moi. L'orientation méthodologique impulsée par la direction privilégiait en outre des correctifs rapides, une manière d'aborder la dette technique qui correspondait moins à la mienne. La proposition de David offrait à l'inverse un projet dont je connaissais la direction et les perspectives, lui aussi centré sur la réduction de la dette technique, et la possibilité de revenir à terme au développement C++ : une perspective qui avait compté dans mon choix initial et que la situation ne garantissait plus.

À la différence de Free Mobile, où mon rôle était transverse et orienté support, j'occupais désormais un poste de développeur travaillant directement sur le fonctionnel. Le contexte restait toutefois marqué par la dette technique : l'enjeu était de demeurer productif face au marché, c'est-à-dire de continuer à livrer de nouvelles fonctionnalités, tout en résorbant progressivement cette dette.

2.3 Positionnement des missions

Sept missions jalonnent ces deux années : cinq chez Free Mobile, deux chez Beemo Technologie, ces dernières étant encore en cours au moment de la rédaction. Plutôt que de les présenter dans l'ordre chronologique, restitué par le diagramme de Gantt en annexe A, ce mémoire les regroupe par nature, en trois chapitres thématiques : **outiller la productivité et standardiser l'environnement de développement**, **rendre testable un logiciel legacy** pour résorber les régressions et s'en prémunir, et **développer du code à visée de production dans un existant endetté**. Le tableau 2.1 en donne la vue d'ensemble.

Thème / Entreprise	**Mission**	**Objectif au regard de la dette technique**
Outils : productivité et standardisation de l'environnement de développement		
Free Mobile	Duplication Grafana	Fiabiliser la réplication de la supervision
Free Mobile	Compilation C++	Maîtriser le coût de compilation (CPU/RAM)
Beemo	Rétro-ing. SDK	Débloquer et documenter une compilation existante
Testabilité d'un logiciel legacy		
Free Mobile	Framework de test	Industrialiser la non-régression
Free Mobile	Extension VS Code	Ouvrir l'usage du framework à toute l'équipe
Free Mobile	Portage multi-forks	Mutualiser la qualité sur les variantes
Code à visée de production dans un existant endetté		
Beemo	Immuabilité	Livrer une fonctionnalité sans accroître la dette

Table 2.1 – Vue d'ensemble des missions, regroupées par nature.

Chapitre 3

Environnement technique

Les environnements techniques de Free Mobile et de Beemo se sont révélés suffisamment proches pour que cette similarité ait précisément motivé ma candidature : c'est ce type d'environnement qui m'intéressait. Tous deux s'organisent autour d'un logiciel principal écrit en C++, qu'entoure un ensemble d'outils satellites. Plutôt que d'en dresser l'inventaire, ce chapitre s'en tient à ce que mes missions ont effectivement mobilisé et renvoie aux travaux pour le détail de chaque usage ; le reste de la pile n'est évoqué qu'en arrière-plan, le temps de situer le terrain.

3.1 Un socle C++ entouré de langages satellites

Dans les deux entreprises, le logiciel central est écrit en C++. Le choix tient d'abord à l'attrait du langage lui-même : il a beaucoup évolué et propose aujourd'hui une bibliothèque standard et des fonctionnalités permettant d'écrire un code conforme aux recommandations issues des dernières avancées du génie logiciel. Étant compilé, il offre en outre des performances élevées : chez Free Mobile, où un gros volume de données devait être traité, ce point était impératif ; chez Beemo, où les *box* ont vocation à sauvegarder plusieurs téraoctets de données, de bonnes performances restent nécessaires pour garantir une expérience utilisateur correcte.

Autour de ce cœur gravitent des langages plus légers, choisis pour leur facilité de déploiement plutôt que pour leur performance : l'absence d'étape de compilation dispense d'une chaîne de Continuous Integration (intégration continue) (CI)/Continuous Delivery / Deployment (CD) pour produire les nombreux outils internes du quotidien. Python y tient le premier rôle, du simple *scripting* appelé en *batch* jusqu'au framework de test d'intégration bâti sur Pytest (section 6.1) ; TypeScript a servi à l'extension *Visual Studio Code* qui l'outille (section 6.2) ; et les scénarios de test eux-mêmes reposent sur un format dérivé du YAML,

complété par des extensions en Lua. C'est cette articulation entre un socle compilé exigeant et une périphérie outillée que mes missions ont eu à traiter.

3.2 Compiler, versionner, intégrer

Dans les deux entreprises, les logiciels C++ sont compilés sous Linux avec la chaîne de compilation (toolchain) GNU (`g++`), mais l'outil qui pilote cette chaîne diffère selon les contextes. Chez Free Mobile, la compilation du *Valo Engine* est orchestrée par **CMake**, qui génère les *makefiles* effectivement exécutés ; la taille de la base de code y rend la compilation parallèle si gourmande en ressources qu'elle devient un sujet d'ingénierie à part entière (section 5.2). Chez Beemo, l'outillage est plus hétérogène : le *Beeserver*, logiciel embarqué dans les *box*, est bâti à partir de *makefiles* écrits à la main, CMake n'étant pas déployable sur une partie du parc qui tourne encore sous une distribution Linux datée ; l'agent ESX, lui, dépend du DSDK (*Daemon SDK*), le SDK propriétaire fourni par VMware (désormais Broadcom) pour développer les *daemons* exécutés sur les hyperviseurs ESXi, qui impose sa propre chaîne de construction (section 5.3). **Docker** complète l'outillage en encapsulant une chaîne de compilation complète dans une image reproductible ; il a notamment servi, chez Beemo, à rapatrier en local un *build* jusque-là confiné à une machine distante.

Le versionnement repose sur **Git**, avec une forge **GitLab** dans les deux entreprises, où les modifications sont intégrées par *merge requests* soumises à revue et où les pipelines de CI sont également hébergés. Ce fonctionnement, aujourd'hui standard, prend ici un relief particulier : une partie des sources du logiciel de valorisation n'était *pas* versionnée à l'arrivée de l'équipe, et certains binaires en production ne correspondaient plus à aucune source identifiée, un cas typique de dette dont la résorption conditionne tout le reste. L'CI prolonge la même logique : les suites de tests, dont le framework de la section 6.1, ont vocation à s'exécuter automatiquement à chaque modification, pour détecter les régressions avant la mise en production plutôt qu'après.

3.3 Stockage et supervision

Le cœur télécom de Free Mobile s'appuie sur un éventail de briques de données : une base MySQL pour les usages historiques liés aux abonnés, Cassandra puis ClickHouse pour le stockage des Call Detail Record (enregistrement détaillé d'appel) (CDR), Apache Kafka pour le transport des flux vers les autres équipes. Ces briques relèvent toutefois du décor de mes missions plutôt que de leur matière. La seule qu'elles touchent directement est la supervision : assurée par **Grafana**, qui agrège dans des tableaux de bord les métriques remontées par le *Valo Engine*, elle constitue le point de départ de ma première tâche

(section 5.1).

3.4 Organisation du travail

Les deux équipes travaillent en méthode *agile*, structurée en *sprints* de deux à trois semaines et rythmée par un point quotidien. Beemo en décline une forme plus complète, avec une phase de raffinement (*refinement*) où les tickets sont précisés avant développement et une rétrospective en fin de sprint ; Free Mobile se montrait plus souple sur ces rituels. Au-delà de ces nuances, la méthode répondait également bien aux besoins des deux contextes.

La vraie différence tenait à mon positionnement. Chez Free Mobile, mes développements portaient sur des outils annexes dont la production ne dépendait pas directement : j'y travaillais « hors sprint », ce qui m'a offert un réel confort en me dégageant de la pression des échéances de livraison. Chez Beemo, mes développements ayant vocation à être livrés en production, j'étais cette fois pleinement intégré aux sprints.

Chapitre 4

Appropriation scientifique : la dette technique et ses enjeux business

4.1 Définition et métaphore

La notion de dette technique naît d'une métaphore financière due à Ward Cunningham [1] : livrer rapidement un code imparfait revient à contracter un emprunt. L'emprunt n'est pas condamnable en soi : il permet d'aller plus vite, de saisir une opportunité de marché, d'apprendre du réel avant de raffiner. Mais comme tout emprunt, il se décompose en un **principal**, c'est-à-dire l'écart entre le code tel qu'il est et le code tel qu'il devrait être, et des **intérêts** : le surcoût payé à chaque intervention ultérieure tant que le principal n'est pas remboursé. Un code mal structuré ne coûte rien tant qu'on n'y touche pas ; il coûte à chaque modification, et ce coût croît avec le temps. Le « remboursement » porte un nom : le *refactoring* [4].

La métaphore a depuis été affinée en un véritable cadre d'analyse [8], dont il faut retenir un point essentiel : toute dette n'est pas une faute. Martin Fowler propose de la classer selon deux axes [3] : la dette peut être **délibérée** ou **accidentelle**, et **prudente** ou **imprudente**. Une équipe qui choisit consciemment une solution rapide pour tenir une échéance, en sachant ce qu'elle devra rembourser, contracte une dette délibérée et prudente, qui constitue un outil de pilotage légitime. La dette dangereuse est celle qui s'accumule à l'insu de tous : accidentelle, par méconnaissance des bonnes pratiques, ou imprudente, par renoncement permanent à tout remboursement. Les contextes rencontrés durant cette alternance relèvent pour l'essentiel de cette seconde catégorie : une dette héritée, non documentée, dont les contracteurs avaient quitté l'entreprise.

4.2 Typologie de la dette technique

Parler de « la » dette technique au singulier masque des réalités très différentes, qui n'appellent pas les mêmes remèdes. Les travaux présentés dans ce mémoire en recoupent cinq formes principales.

Dette de code. La plus visible : fonctions interminables, abus de directives de préprocesseur, constructions trahissant une maîtrise approximative du langage. C'est l'état du firmware de la Beebox chez Beemo (section 7.1).

Dette de test. L'absence de filet de sécurité automatisé : tests unitaires, d'intégration, de non-régression. C'est la forme dominante rencontrée chez Free Mobile, et la cible directe des sections 6.1 à 6.3.

Dette d'architecture. Des choix structurels qui freinent toute évolution : le couplage massif induit par un en-tête incluant tout le projet (section 7.1), ou la multiplication de fork divergents là où un socle commun aurait dû être maintenu (section 6.3).

Dette de documentation et de connaissance. Le savoir-faire qui ne réside que dans la tête d'une personne : la procédure de compilation de l'agent ESX ne survivait que par son unique détenteur (section 5.3), et une partie des sources du moteur de valorisation n'était pas même versionnée.

Dette de build et de dépendances. Une chaîne de production opaque, non reproductible ou devenue insoutenable : la saturation mémoire de la compilation parallèle (section 5.2) et le *build* distant, manuel et non documenté de l'agent ESX (section 5.3).

Cette typologie éclaire un point de méthode : la dette de test et la dette de connaissance sont les plus sournoises, car elles ne se manifestent par aucun symptôme dans le code lui-même : elles ne se révèlent qu'au moment où l'on modifie le logiciel, ou au départ d'un collaborateur.

4.3 Enjeux business

Traduire la dette technique en termes d'entreprise revient d'abord à reconnaître une tension difficilement contournable. Du strict point de vue du développeur, le confort absolu dans le traitement de la dette consisterait à suspendre tout développement de nouvelle fonctionnalité *business* pour se consacrer pleinement à l'assainissement du code existant. Cette option est pourtant illusoire : il est impossible d'ignorer la nécessité de continuer à livrer de nouvelles fonctionnalités, ne serait-ce que pour rester compétitif. Le marché du numérique est en effet particulièrement volatil et changeant, et cette contrainte s'impose quelle que soit la taille de l'entreprise. Un gel complet des développements serait, au

mieux, financièrement encaissable par les plus grandes structures, au prix d'un impact non négligeable sur leur croissance ; il serait tout simplement rédhibitoire pour celles qui sont incapables de mener de front maintenance du legacy et nouveaux développements.

La dette technique se pose donc rarement comme un problème purement technique : elle relève d'un arbitrage permanent entre le temps consacré à la production de valeur immédiate (le *time-to-market* des fonctionnalités attendues par le marché) et celui investi dans la soutenabilité du logiciel à long terme : coût de maintenance, risque de régression et fiabilité, cette dernière pesant tout particulièrement pour un opérateur télécom. La littérature souligne d'ailleurs que la dette réellement *mesurée* dans le code excède fréquemment celle que les équipes *reconnaissent* [10], et propose des modèles explicatifs de sa production dans les contextes Agile et DevOps [7]. Toute la difficulté consiste à résorber la dette *sans* renoncer à cette capacité de livraison continue. C'est précisément cet équilibre que les deux entreprises ont eu à construire, de manière empirique, comme le détaille le retour d'expérience qui suit.

4.4 Retour d'expérience : stratégies d'adressage et dimension sociale

Au-delà des leviers strictement techniques, les deux entreprises où j'ai exercé montrent que la manière d'adresser la dette dépend autant de l'état de la base de code que de contraintes économiques, mais aussi humaines. Cette section revient sur ce que j'ai pu observer dans chacun de ces contextes.

4.4.1 Une même logique d'adressage incrémental

Face à la tension décrite plus haut, les deux entreprises ont adopté une même réponse : un adressage incrémental de la dette. Plutôt que de geler les développements, il s'agit de partir du code existant et, lorsqu'une fonctionnalité doit être ajoutée à un endroit donné, de profiter de l'intervention pour réécrire le code concerné lorsque cela est possible. Chez Free Mobile, cette logique a résulté d'une évolution de la stratégie, privilégiant cette réécriture « au fil de l'eau » plutôt qu'une reprise de contrôle purement frontale.

4.4.2 Des dettes de nature différente

Si la logique est commune, la dette ne s'exprimait pas de la même façon d'une entreprise à l'autre, ce qui conditionnait la marge de manœuvre disponible.

Chez Free Mobile, la dette tenait surtout à la multiplication de *forks* dotés de spécificités, vraisemblablement créés faute de temps pour produire de nouvelles versions. Le code restait

toutefois globalement compréhensible et logique, malgré certains choix architecturaux discutables et une méconnaissance de quelques outils externes : on y recourait par exemple à des requêtes SQL là où des mécanismes plus adaptés existaient.

Chez Beemo, les contraintes de réécriture étaient bien plus fortes. Le code y est plus cloisonné et, paradoxalement, plus lisible, mais aussi beaucoup plus ancien : écrit à l'origine en C, puis en C++. Pour composer avec ces contraintes, des *patterns* ont été mis en place afin de cloisonner totalement le code *legacy* du code moderne : les nouvelles fonctionnalités sont écrites dans une librairie embarquée puis injectée dans le code historique. Le développement suit alors un cours classique ; lorsqu'une fonctionnalité doit être modernisée en profondeur, elle est réécrite côté moderne, puis « gommée » du code ancien. Cette étape de gommage n'a pas encore été engagée frontalement, afin de préserver une compatibilité permettant le *merge* et le rapatriement de branches *legacy* porteuses de correctifs non encore intégrés.

4.4.3 L'enjeu social du traitement de la dette

L'enjeu qui m'a le plus marqué est toutefois d'ordre social. Cette dimension est d'ailleurs reconnue par la littérature, qui a étendu la métaphore en une « dette sociale » : des structures d'équipe et de communication sous-optimales qui, comme la dette du code, génèrent un surcoût croissant si elles ne sont pas traitées [13]. La phase de reprise de contrôle totale du logiciel s'est révélée très chronophage et, pour être honnête, peu gratifiante pour les développeurs. J'ai ainsi pu observer chez certains collègues une perte de motivation parfaitement compréhensible : si la technique constitue un domaine en soi très intéressant, passer des heures dans du code *legacy* peu agréable à lire, pour n'y greffer que de petits ajouts au sein d'un existant déjà complexe, n'a rien de particulièrement attrayant. Ce ressenti est lui aussi documenté empiriquement : une expérience de psychométrie appliquée au génie logiciel montre que la confrontation à un code endetté provoque chez les développeurs des états affectifs négatifs mesurables (déplaisir, anxiété) pouvant aller jusqu'à la réticence à intervenir dans le code concerné [9].

Ce constat n'est pas anecdotique : pour qu'une dette technique soit correctement résorbée, l'enjeu humain, c'est-à-dire la motivation des équipes et la reconnaissance d'un travail ingrat, me paraît, au regard de ces deux années d'alternance, tout aussi déterminant que les choix d'outillage ou d'architecture.

4.5 Leviers de réduction

Face à ces différentes formes de dette, la littérature et la pratique convergent vers un petit nombre de leviers complémentaires, chacun étant actionné par au moins une des missions

présentées dans la suite.

Le premier levier est le **test automatisé**. Michael Feathers définit d'ailleurs le code *legacy* non par son âge, mais par l'absence de tests [2] : sans filet de non-régression, toute modification est un pari, et la peur de casser fige le logiciel dans son état. Doter un code existant de tests est donc le préalable à tout remboursement de dette ; c'est l'objet même du framework de test et de son portage vers les forks legacy (sections 6.1 et 6.3).

Le deuxième est le **refactoring**, c'est-à-dire l'amélioration de la structure du code à comportement constant [4]. Il suppose le premier levier : on ne réécrit sereinement que ce que l'on peut re-tester. Lorsque le refactoring frontal est trop risqué, des stratégies d'isolation, consistant à construire du code moderne à côté du legacy derrière une frontière nette, permettent d'avancer sans tout reprendre (section 7.1).

Le troisième est l'**automatisation de la chaîne de livraison** : intégration et déploiement continus [6], mais aussi, plus en amont, une chaîne de compilation soutenable et reproductible. Un *build* qui échoue par saturation mémoire (section 5.2) ou qui ne fonctionne que sur une machine distante non documentée (section 5.3) constitue un goulot d'étranglement qui renchérit chaque livraison.

Le dernier levier, transversal, est l'**outillage et la capitalisation de la connaissance** : automatiser les procédures manuelles (section 5.1), versionner ce qui ne l'était pas, documenter sous forme exécutable (un patch versionné vaut mieux qu'une procédure orale) et abaisser la barrière d'entrée des outils de qualité pour qu'ils soient réellement adoptés (section 6.2). Ce levier répond directement à la dette de connaissance, la plus exposée au départ des personnes.

4.6 Grille de lecture des missions

Les missions présentées dans la suite partagent un contexte commun, qui en constitue le fil conducteur : l'absence de tests, qu'ils soient unitaires ou d'intégration. À l'origine, ces logiciels n'en comportaient aucun ; à mon arrivée chez Free Mobile, l'équipe montée par David Desmarest en avait certes introduit quelques-uns, mais la couverture demeurait globalement insuffisante.

Cette dette de test se double d'une connaissance fonctionnelle souvent parcellaire des logiciels appelés à évoluer. La conjonction de ces deux facteurs expose à de forts risques de régression dès qu'une modification est apportée : c'est précisément ce qui justifie la mise en place d'une stratégie de tests et qui rend, plus largement, le développement de nouvelles fonctionnalités difficile. Chacune des tâches qui suivent peut ainsi se lire comme une réponse à une facette de ce constat.

Les trois chapitres suivants présentent ces missions regroupées par nature, selon les leviers qu'elles actionnent : **outiller la productivité et standardiser l'environnement de développement** (chapitre 5), **rendre testable un logiciel legacy** pour résorber les régressions et s'en prémunir (chapitre 6), puis **développer du code à visée de production dans un existant endetté** (chapitre 7).

Chapitre 5

Outiller la productivité et standardiser l'environnement de développement

Ce premier chapitre regroupe trois missions qui partagent une même nature : construire ou reprendre en main l'**outillage** qui conditionne le travail quotidien des développeurs. Relues à travers la typologie du chapitre 4, elles s'attaquent principalement à la **dette de build et de dépendances** et à la **dette de connaissance** : des procédures manuelles, non reproductibles ou détenues par une seule personne, qui renchérissent chaque intervention sur le logiciel.

L'intérêt de **standardiser l'environnement de développement** est double. C'est d'abord un enjeu de **productivité** : tout ce qu'un développeur doit refaire à la main (répliquer une configuration de supervision, subir une compilation qui sature sa machine, pousser le code vers une machine distante pour le construire) est un coût payé à chaque itération, par chaque membre de l'équipe ; l'outiller, c'est le payer une fois. Ce coût se chiffre : à l'échelle d'une journée de développeur, qui recompile et redéploie des dizaines de fois, une à deux minutes gagnées par cycle se comptent presque en **heures** récupérées, soit du temps passé à produire plutôt qu'à attendre. C'est ensuite un enjeu de **capitalisation** : un environnement standardisé transforme un savoir-faire individuel en savoir *exécutable* (outil, configuration versionnée, image de conteneur) qui survit au départ des personnes, là où la procédure manuelle disparaît avec son détenteur. Enfin, un socle commun à toute l'équipe supprime les écarts entre postes (« ça compile chez moi ») qui transforment chaque anomalie en enquête.

Trois missions actionnent ce levier : un outil de duplication de la configuration Grafana (section 5.1) et la maîtrise du coût de compilation du *Valo Engine* (section 5.2), toutes deux chez Free Mobile, puis, chez Beemo, la rétro-ingénierie du DSDK pour rendre le *build* de l'agent ESX local, reproductible et documenté (section 5.3).

5.1 Un outil de duplication de configuration Grafana

5.1.1 Contexte et dette technique initiale

La supervision de la valorisation s'appuie sur **Grafana**, qui agrège dans des *dashboards* les métriques remontées par le *Valo Engine*. Or cette supervision n'existe pas sur une seule instance : on en retrouve plusieurs, typiquement une par environnement (développement, recette, production) ou par réseau, qui doivent présenter les **mêmes tableaux de bord**. Faire vivre ces instances en parallèle suppose donc de **répliquer** la configuration de l'une vers les autres à chaque évolution.

Cette réplication ne se résume pas à une copie : chaque tableau de bord référence ses **sources de données** (*datasources*) par un **identifiant propre à l'instance**. Le même tableau, copié tel quel d'une instance à l'autre, pointe alors vers des identifiants qui n'y existent pas, et l'affichage se retrouve cassé. La synchronisation manuelle (export, copie, puis correction au cas par cas des identifiants) était donc à la fois **fastidieuse et source d'erreurs** : une dette d'outillage sur la chaîne de supervision elle-même.

5.1.2 Objectif de la mission

La mission consistait à **reprendre le travail d'un stagiaire**, qui avait amorcé un Proof of Concept (PoC) sur ce sujet, pour en faire un outil exploitable. Le but : **dupliquer l'ensemble des tableaux de bord** d'une instance Grafana vers une autre, puis **réécrire automatiquement** les identifiants de sources de données pour qu'ils correspondent à ceux de l'instance cible. Le cœur du problème est donc un **appariement** : faire correspondre, deux à deux, les sources de données de l'instance d'origine et celles de l'instance de destination, afin de substituer chaque identifiant par son équivalent.

5.1.3 Conception et choix techniques

La logique métier est directe : Grafana expose une Application Programming Interface (API) HTTP permettant de **lister** les tableaux de bord et les sources de données d'une instance, puis d'y **créer** de nouveaux objets. L'outil interroge l'instance source, construit une **table de correspondance** entre les sources de données des deux instances (appariées par nom), puis parcourt chaque tableau de bord pour y **remplacer** les identifiants avant de le pousser sur l'instance cible.

L'enjeu de conception portait surtout sur l'**interface**. Plutôt qu'une ligne de commande aride, l'outil a été doté d'une **interface en mode texte** (Text User Interface (interface en mode texte) (TUI)) à l'aide du framework Python **Textual** [14]. Ce framework a été une **découverte marquante** : il permet de décrire une interface terminal selon un modèle très

proche du développement web, avec une structure de composants apparentée au *HTML*, mise en forme par des feuilles de style apparentées au *CSS*. On obtient ainsi, dans un simple terminal, une interface interactive (listes, sélection des instances, validation de l'appariement) sans quitter l'environnement de travail des administrateurs.

5.1.4 Réalisation

Le cœur de l'outil tient dans la réécriture des identifiants : à partir de la table de correspondance construite entre les deux instances, chaque référence à une source de données présente dans un tableau de bord est substituée par son équivalent côté cible.

5.1.5 Difficultés rencontrées

S'agissant de ma **première tâche** dans l'équipe, l'enjeu principal n'était pas tant la complexité, le besoin étant circonscrit et la mission relativement **rapide**, que la **prise en main** de l'environnement : le mode de travail de l'équipe, le dépôt, et la reprise d'un PoC déjà entamé par un tiers. La principale subtilité technique tenait à la **robustesse de l'appariement** des sources de données, l'appariement par nom supposant une nomenclature cohérente d'une instance à l'autre.

5.1.6 Résultats et bénéfice business

L'outil a transformé une synchronisation manuelle et faillible en une opération **automatisée et reproductible** : là où la procédure exigeait d'exporter, copier puis corriger chaque tableau de bord à la main, la duplication complète d'une instance se réduit désormais à l'exécution de l'outil. La classe d'erreurs liée aux identifiants de sources de données mal réécrits disparaît purement et simplement, et avec elle les tableaux de bord silencieusement cassés sur les instances secondaires, c'est-à-dire des angles morts de supervision sur un logiciel critique.

Au regard du fil rouge de ce mémoire (chapitre 4), cette mission illustre le levier de l'**outillage** : une dette d'exploitation modeste en apparence, mais remboursée une fois pour toutes par l'automatisation. Au-delà du livrable, elle a joué un rôle de **mise en jambe** : prise de contact avec l'équipe et son environnement, et découverte de Textual, un framework réinvesti par la suite.

5.2 Maîtrise du coût de compilation du Valo Engine

5.2.1 Contexte et dette technique initiale

Le *Valo Engine* est écrit en **C++**, langage dont la compilation est réputée **coûteuse**, en temps de calcul comme en mémoire. Cette lourdeur tient à son modèle historique de compilation, hérité du C : chaque fichier source est compilé en une **unité de traduction** indépendante, qui inclut *textuellement* les en-têtes (*headers*) dont elle dépend. Les **templates** aggravent ce mécanisme : leur définition résidant dans les en-têtes, ils sont **réinstanciés à chaque unité de traduction** qui les inclut. Une même instanciation est ainsi recompilée des dizaines de fois à travers le projet.

À strictement parler, ce coût ne devient un problème que parce que l'on cherche à compiler **en parallèle** : une compilation séquentielle resterait sobre en mémoire, mais à un prix prohibitif en temps. Or y renoncer n'est pas une option : développer en C++, c'est s'appuyer sur le compilateur pour se prémunir d'un maximum d'erreurs *avant même* d'exécuter le programme, et la rapidité du *build* conditionne donc toute la boucle de développement (lancer les tests unitaires, valider une modification), avec les gains de temps évoqués en introduction de ce chapitre.

Le pic se produit à l'**édition des liens**. L'éditeur de liens doit y rassembler tous les fichiers objets et **dédupliquer** les définitions redondantes d'une même fonction ou instanciation, issues des multiples unités de traduction ; l'empreinte mémoire de cette étape explose à mesure que le nombre de symboles croît. En lançant, via les *makefiles* générés par **CMake**, une compilation parallèle exploitant l'ensemble des cœurs d'une machine, de l'ordre de **24**, la chaîne de compilation (toolchain) **saturait la RAM** précisément à cette phase de liaison, rendant la compilation instable voire impossible à mener à son terme. La dette était ici nichée dans la **chaîne de compilation** elle-même : un atout de productivité (le parallélisme) se retournait en obstacle.

5.2.2 Objectif de la mission

La mission consistait à étudier la **faisabilité d'un compromis** : exploiter au mieux les cœurs disponibles *sans* provoquer la saturation mémoire qui faisait échouer la compilation. Autrement dit, **ordonnancer** la compilation pour qu'elle reste **soutenable** en mémoire, en particulier durant l'édition des liens, tout en conservant un parallélisme aussi élevé que le permet le budget RAM de la machine.

5.2.3 Conception et choix techniques

Le diagnostic a d'abord consisté à **isoler la phase critique**. La compilation des unités de traduction se parallélise sans difficulté majeure ; c'est l'**édition des liens**, beaucoup plus gourmande en mémoire par processus, qui fait diverger la consommation lorsque plusieurs liaisons s'exécutent simultanément. La piste retenue revient donc à **découpler le degré de parallélisme** des deux phases : conserver un parallélisme large pour la compilation, mais **borner** le nombre de tâches d'édition des liens exécutées de front, afin de plafonner le pic mémoire sous la capacité réelle de la machine.

À plus long terme, la **cause racine**, à savoir la recompilation répétée des en-têtes et des templates, trouve une réponse dans les **modules** de **C++20**, qui compilent une interface **une seule fois** au lieu de la réanalyser à chaque inclusion. Cette voie n'était toutefois pas exploitable : **le support des modules reste partiel** dans les compilateurs, et surtout ceux-ci continuent de retenir un standard antérieur *par défaut*. À titre de repère, sur Ubuntu 24.04 LTS, `g++` 13.3 conserve **C++17** (`__cplusplus = 201703L`) ; et cette valeur n'a rien d'une survivance de l'environnement local, puisque les versions les plus récentes de la chaîne GNU conservent ce même défaut : `g++` 15 retient toujours `-std=gnu++17`, le basculement vers C++20 par défaut n'étant annoncé que pour `g++` 16 [5]. L'adoption de C++20 ou C++23, et de leurs modules, supposerait donc une migration explicite de la chaîne de compilation (toolchain) qui n'est pas encore une option standard et mûre.

5.2.4 Réalisation

L'idée se matérialise en **limitant la concurrence de l'étape de liaison** indépendamment de celle de la compilation, de sorte que le pic mémoire cumulé des éditions de liens reste sous le plafond de la machine.

5.2.5 Difficultés rencontrées

La principale difficulté a été de **caractériser le pic mémoire** : la saturation ne survenait qu'à l'édition des liens et seulement sous forte concurrence, ce qui en rendait l'observation peu intuitive : le coût n'est pas réparti uniformément sur la durée de la compilation, mais **concentré** sur sa phase finale. La seconde tenait à la **tension entre vitesse et mémoire** : tout plafonnement de la concurrence de liaison améliore la stabilité au prix d'un allongement du temps de compilation, et le bon réglage dépend étroitement du couple (mémoire disponible, taille des objets à lier).

5.2.6 Résultats et bénéfice business

En bornant la concurrence de la phase de liaison, la compilation redevient **soutenable** sur les machines de développement : elle peut être menée à son terme sans saturer la RAM, là où une parallélisation naïve la faisait échouer, tout en conservant un parallélisme maximal sur la phase de compilation proprement dite, qui représente l'essentiel de la durée totale. Pour l'équipe, le bénéfice est direct : un *build* fiable est la condition de toute boucle de développement courte, et donc de toute intervention sereine sur le code.

Au-delà du résultat immédiat, cette mission a été l'occasion de **mettre au jour une dette de fond**, le coût du modèle de compilation C++ préalable aux modules, dont la résorption complète reste suspendue à la maturation du support de C++20/23 dans les compilateurs.

5.3 Rétro-ingénierie pour la compilation avec un SDK

5.3.1 Contexte et dette technique initiale

Première mission menée chez **Beemo Technologie** (présentée au chapitre 2), cette tâche est sensiblement plus **courte** que les précédentes, mais n'en est pas moins **déterminante** : elle porte sur l'un des logiciels de Beemo, **l'agent ESX**, et vise à reprendre le contrôle sur une partie de sa **chaîne de production**. Sa portée dépasse le simple correctif : il s'agit d'**automatiser** et d'**harmoniser** la méthode de fabrication de cet agent, ce qui la rend, à mes yeux, aussi structurante que la réalisation du framework de test étalée sur près d'un an et demi.

Avant cette mission, mettre à jour l'agent ESX supposait une manipulation manuelle et fragile : on **poussait la base de code vers une machine distante**, sur laquelle elle était compilée avec le **DSDK** (*Daemon SDK*), le SDK fourni par VMware (désormais Broadcom) pour développer les *daemons* (services d'arrière-plan) exécutés sur les hyperviseurs ESXi, avant de pouvoir être testée.

La dette technique est apparue avec le **passage à la version 8 du DSDK**, nécessaire pour supporter la version 8 de l'agent. Dans sa configuration de base, le DSDK impose le flag de compilation C++ `-Wall -Werror`, qui **convertit l'ensemble des avertissements en erreurs** bloquantes. Or l'agent est écrit en C++11, avec une **absence quasi totale de bonnes pratiques** de génie logiciel : certains fichiers dépassent les **50 000 lignes**. Le volume d'avertissements produit est dès lors **colossal** et intraitable en l'état : il met un frein considérable à la capacité de l'équipe à **livrer des mises à jour en continu**.

Une rustine existait bien (des modifications appliquées directement sur la machine de compilation pour neutraliser ce comportement), mais elle n'était **ni reproductible ni documentée**. Ce savoir reposait sur le développeur **que je remplace** : son départ menaçait

de l'emporter avec lui.

5.3.2 Objectif de la mission

La mission a poursuivi deux objectifs complémentaires :

- **s'affranchir de la machine de compilation distante** en rendant le *build* reproductible localement, à l'aide d'un conteneur **Docker** embarquant la chaîne de compilation (toolchain) du DSDK ;
- **rendre les correctifs du SDK reproductibles et documentés**, en substituant aux modifications manuelles des **fichiers de patch** versionnés, appliqués automatiquement lors de la construction de l'image.

Atteindre ces deux objectifs supposait au préalable de **comprendre le fonctionnement interne du DSDK** : une démarche de rétro-ingénierie s'imposait, le SDK n'exposant aucun moyen standard d'agir sur ses flags de compilation.

5.3.3 Démarche de rétro-ingénierie

Le cœur de la tâche a consisté à **retrouver où et comment le DSDK injecte ses flags de compilation**. Cette mécanique n'est **pas exposée** par le SDK : il n'existe aucune option officielle pour assouplir le couple `-Wall -Werror`. Le DSDK reposant sur le système de construction **SCons**, il a fallu **rétro-concevoir ses fichiers de build** pour localiser le point précis où ces flags sont posés, puis déterminer comment les neutraliser sans dénaturer le reste de la chaîne de compilation (toolchain).

La difficulté principale n'a pas été d'identifier la modification à apporter, mais de **trouver le bon point d'injection** : le DSDK **n'est pas conçu pour être modifié**, et un patch mal placé compromet l'intégrité de la chaîne. Une fois ce point localisé, la suite s'est révélée nettement plus simple : il a suffi de cibler les fichiers concernés et d'en dériver des correctifs reproductibles.

5.3.4 Réalisation

La solution repose sur un **Dockerfile** qui reconstitue l'environnement de compilation auparavant confiné à la machine distante, puis applique les correctifs du SDK de façon déterministe. Chaque modification est isolée dans un fichier `.patch` versionné et appliquée via la commande `patch` lors de la construction de l'image, là où elle était jusqu'alors saisie à la main, sans trace.

5.3.5 Difficultés rencontrées

La principale difficulté a tenu à l'**opacité du DSDK** et à l'identification du point d'injection des patchs, évoquée plus haut. Une seconde est apparue en aval : l'un des correctifs appliqués au SDK a eu un **effet de bord sur la certification** de l'un des *packages* générés, qui n'est plus accepté en l'état. C'est la raison pour laquelle la tâche demeure **encore en cours** : le *build* local et reproductible est acquis, mais la **validation du package** reste à fiabiliser.

5.3.6 Résultats et bénéfice business

La mission a transformé une **procédure manuelle, distante et non documentée** en un *build* **local, reproductible et versionné**. Les correctifs du SDK, jadis tributaires d'un seul développeur, sont désormais **capitalisés sous forme de patchs** : leur logique survit à tout départ, ce qui répond directement au risque de **perte de connaissance** à l'origine de la dette. En supprimant le frein imposé par `-Wall -Werror`, l'équipe retrouve sa capacité à **livrer des mises à jour de l'agent ESX en continu**.

Le chantier restant, à savoir résoudre l'impact sur la certification, s'inscrit dans une perspective plus large : **automatiser le processus de validation d'un package par Broadcom**, aujourd'hui **très chronophage**. À terme, c'est l'ensemble de la chaîne de production de l'agent qui gagnerait en **harmonisation** et en **autonomie**, prolongeant la résorption de la dette bien au-delà du seul déblocage de la compilation.

Chapitre 6

Rendre testable un logiciel legacy : résorber les régressions et s'en prémunir

Ce deuxième chapitre nous ramène chez Free Mobile et rassemble les trois missions consacrées à la **testabilité** du *Valo Engine* et de ses forks. Il répond à la forme de dette dominante identifiée au chapitre 4 : la **dette de test**, c'est-à-dire l'absence de filet de sécurité automatisé autour d'un logiciel pourtant critique pour la facturation : le *Valo Engine* calcule le coût de chaque appel, et une régression ou une panne sur ce maillon peut engendrer des pertes **de l'ordre du million d'euros en une semaine**.

L'importance des tests dans un contexte legacy dépasse la seule détection d'anomalies. Sans filet de non-régression, chaque modification (correctif, évolution, et à plus forte raison refactoring) est un pari : la seule validation possible est manuelle, donc partielle, coûteuse et non rejouable. Les régressions se découvrent alors en production, là où leur coût est maximal. Surtout, la dette de test verrouille toutes les autres : on ne peut résorber une dette de code ou d'architecture qu'en modifiant le logiciel, ce que l'on ne s'autorise pas sans filet. Rendre un logiciel legacy testable est donc le **prérequis de tout remboursement** : c'est l'investissement qui rend les autres possibles.

Encore faut-il que ce filet soit fiable et adopté. Les trois missions de ce chapitre s'enchaînent dans cette logique : concevoir un framework de test d'intégration et de non-régression à scénarios déclaratifs (section 6.1), lever les freins à son adoption par l'équipe grâce à une extension VS Code dédiée (section 6.2), puis étendre ce filet aux six forks legacy du moteur par un portage de l'architecture du framework (section 6.3).

6.1 Un framework de test d'intégration et de non-régression

6.1.1 Contexte et dette technique initiale

La non-régression du *Valo Engine* reposait initialement sur une suite de tests écrits en Lua. Ce langage permettait de scripter des scénarios, mais ne s'avérait pas le mieux adapté à ce que l'on cherchait à faire. Sa nature **impérative** laissait en effet au rédacteur d'un test toute latitude pour y introduire de la logique : un test pouvait dès lors échouer non à cause d'une régression du logiciel testé, mais à cause d'un défaut du test lui-même. La frontière entre code de test et code testé devenait floue, ce qui fragilisait la confiance accordée à ce filet de sécurité. À cela s'ajoutait un couplage de bas niveau : le protocole de communication du moteur, un protocole TCP décrit en C, était embarqué dans le Lua, ce qui rendait la suite délicate à maintenir.

6.1.2 Objectif de la mission

L'objectif était de remplacer cette suite Lua par un framework dédié, reposant sur des scénarios **déclaratifs**, volontairement plus contraignants. En restreignant fortement la forme des tests (un scénario décrit *quoi* vérifier, jamais *comment*), on garantit que l'échec d'un test, dès lors que le framework se trouve dans une version stable, provient bien du code du logiciel testé et non de la mécanique de test. Le nouveau framework devait au minimum couvrir le périmètre fonctionnel déjà assuré par le Lua : la simulation des trames d'appel (appels voix, envoi de SMS et de MMS, consommation de data) ainsi que la vérification des données ainsi produites.

6.1.3 Conception et choix techniques

La première étape a consisté à **porter en Python** le protocole TCP par lequel le framework dialogue avec le *Valo Engine*, jusque-là décrit en C et embarqué dans le Lua. Le module `ctypes` de Python permet de définir des structures dont la disposition binaire est strictement équivalente à celle de leurs homologues C ; on reconstitue ainsi un échange de trames identique à celui du moteur sans recourir au langage natif.

Le framework prend en entrée des scénarios de test rédigés en YAML. Ces scénarios sont analysés par un **parseur SAX**, qui présente deux avantages : il permet de récupérer la position (ligne) de chaque instruction, qui est transmise au débogueur pour offrir une exécution *pas à pas*, et il interrompt l'analyse dès la première erreur de syntaxe en indiquant sa localisation et sa nature (arguments mal formés ou instruction non supportée).

L'exécution repose sur une **boucle séquentielle** simple : les instructions sont jouées les unes après les autres, sans branchement ni retour arrière. Chaque ligne de texte est transformée

en objet *instruction* suivant le patron *Command* : une *factory* instancie la commande correspondante à partir d'un *registry* où sont enregistrées toutes les commandes supportées.

Autour de cette boucle, le framework fournit les services nécessaires pour **moquer** les briques avec lesquelles le logiciel testé, le *Valo Engine*, interagit : un service simulant les équipements qui interrogent le moteur (autorisation d'appel ou de SMS, vérification du crédit selon la localisation) et un service recevant les notifications de seuil émises par le moteur (courriels et messages). Enfin, un ensemble d'instructions de **vérification** contrôle, pour une trame d'appel donnée, la cohérence des données générées en base.

Le mécanisme de vérification reprend le principe de *snapshot* déjà employé côté Lua : les actions jouées ainsi que l'état final de la base de données sont sérialisés, puis comparés à une trace de référence enregistrée lors d'une exécution validée ; toute divergence signale une régression. Pour disposer d'emblée d'une restitution des résultats exploitable (comptes rendus, agrégation des échecs, comparable à celle d'une suite de tests classique), le framework a été bâti comme une **surcouche de Pytest** [11], la génération et la comparaison des snapshots étant déléguées à la bibliothèque **syrupy**.

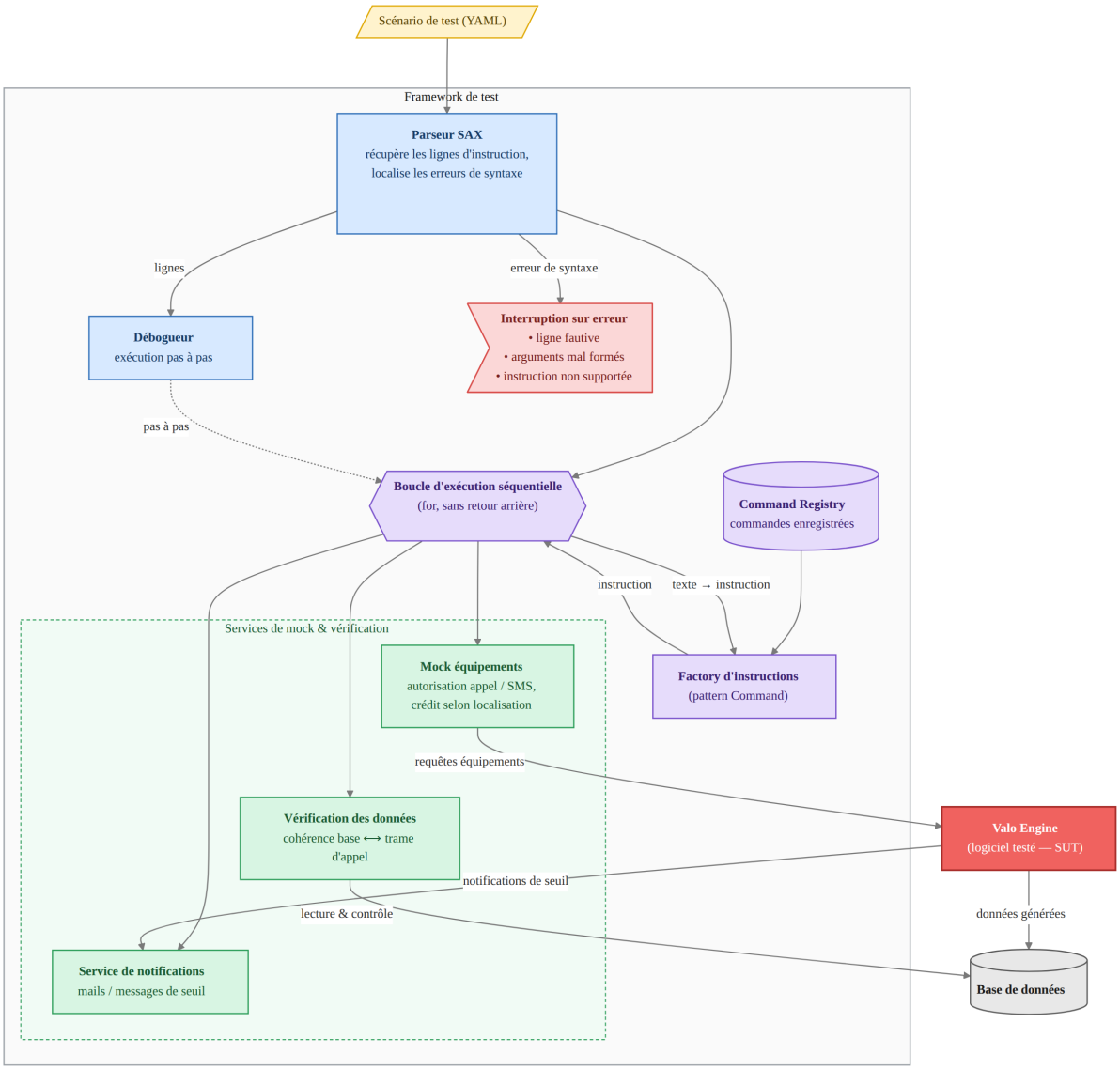


Figure 6.1 – Architecture du framework de test.

6.1.4 Réalisation

Du point de vue du rédacteur de test, tout tient dans le scénario YAML : une suite d'instructions déclaratives, jouées séquentiellement, qui décrivent un flux d'appel et les vérifications attendues. L'extrait ci-dessous (listing 6.1) illustre un scénario d'appel voix national sortant : on y déclare les données de départ (client, forfaits, localisations), puis on enchaîne la mise en place du contexte (`setup`), l'appel (`voice_call`) et les vérifications attendues (`check`).

```yaml
customers:
  - id: 81
    msisdn: 33612345005
    imsi: 2015012345005
    wallet: 5000
    default_package: 1021
    packages:
      - id: 1067
        current_value: 12345
        expiration_date: "2028-12-01T12:00:00Z"

localizations:
  france:
    vlr: 33695029402
  australia:
    vlr: 61

voice_nat_mo:
  - setup:
      location: $france
      customer: 81
      variables:
        call_duration: 24s
        reservation_duration: 1mn
  - voice_call:
      called_number: 336518995221
      duration: $call_duration
      reservation: $reservation_duration
  - check:
      - last-engine-status: TERMINATE_SUCCESS
      - interval:
      - cdr:
          charge: 2cts
```

Listing 6.1 – Scénario de test d'un appel voix national sortant.

On note la concision du format : la variable `$france` ou `$call_duration` référence une donnée déclarée plus haut, et le bloc `check` ne contient que des constats (statut du moteur, intervalle, ligne de facturation à 2 centimes) sans la moindre logique d'assertion.

Côté framework, chaque nom d'instruction est résolu via le *registry* des commandes : la *factory* instancie l'objet *Command* correspondant, qui encapsule l'envoi des trames TCP au *Valo Engine*, reconstituées en Python grâce aux structures `ctypes` équivalentes aux structures C du protocole. L'inscription au *registry* est purement déclarative : un décorateur `@instruction` associe une classe à son nom de scénario (et à ses éventuels alias hérités du Lua), de sorte qu'ajouter une instruction au langage se résume à écrire

une classe (listing 6.2).

```
"""
Play a voice_call between a User and someone for a certain duration
"""

@instruction("voice-call", legacy=["voice_call"])
class VoiceCall(Instruction[dict[str, Any]]):

    def __init__(self, ref: Mark, options: dict[str, Any]):
        super().__init__(ref, options)

    def exec(self, context: BaseContext):
        # Run the logic behind a voice-call
        return
```

Listing 6.2 – Squelette d'une instruction du framework (patron *Command*).

Le décorateur `@instruction("voice-call", legacy=["voice_call"])` enregistre la classe sous son nom courant tout en conservant l'alias `voice_call` employé par les anciens scénarios ; le paramètre `ref` (de type `Mark`) porte la position en ligne issue du parseur SAX, exploitée par le débogueur, et `options` reçoit les arguments déclarés dans le YAML. Toute la logique propre à l'instruction se concentre dans la méthode `exec`.

Le décorateur lui-même (listing 6.3) se réduit à peupler un dictionnaire global, le *registry*, qui associe chaque nom d'instruction à sa classe. Une assertion vérifie au passage que toute classe décorée hérite bien de `Instruction`, garde-fou qui transforme une erreur d'extension du framework en message explicite plutôt qu'en défaillance silencieuse à l'exécution.

```python
instruction_registry: dict[str, type[Instruction[Any]]] = {}

T = TypeVar("T")


def instruction(name: str, legacy: Optional[list[str]] = None):

    def annotation(cls: type[T]) -> type[T]:
        casted_cls = cast(Any, cls)
        assert issubclass(
            casted_cls, Instruction
        ), f"Framework extension error : {cls.__name__} doesn't extends
           Instruction"

        casted_cls.YAML_TAG = name
        instruction_registry[name] = casted_cls
        # ... enregistrement des alias "legacy" (extrait tronque)

        return cls

    return annotation
```

Listing 6.3 – Le décorateur `@instruction` : enregistrement d'une commande dans le *registry*.

Les instructions de vérification, elles, ne contiennent aucune assertion explicite : elles sérialisent l'état observé (actions jouées, contenu de la base), que syrupy compare au *snapshot* de référence. Un test échoue donc pour une raison et une seule : l'état produit par le moteur a divergé de l'état validé, c'est-à-dire une régression.

6.1.5 Difficultés rencontrées

S'agissant de l'une des premières missions d'envergure qui m'étaient confiées, la principale difficulté a tenu au **découpage du travail**. Plutôt que de procéder par versions intermédiaires, livrables et vérifiables au fur et à mesure, je me suis engagé d'emblée dans la conception complète d'une première version couvrant l'ensemble des fonctionnalités du Lua. Ce choix a allongé le délai avant d'obtenir un premier retour concret et m'a privé des points de contrôle qu'un découpage plus fin aurait offerts.

Le second point, apparu avec le recul, concerne le choix de bâtir le framework sur Pytest. Si cet outil fournit gratuitement un socle d'exécution et de reporting éprouvé, il recèle aussi de nombreux mécanismes implicites, sa « *black magic* », dont le comportement peut se révéler déroutant et compliquer le développement d'une surcouche.

6.1.6 Résultats et bénéfice business

Le framework a remplacé la suite Lua sur l'ensemble de son périmètre fonctionnel (appels voix, SMS, MMS, consommation de data et vérification des données produites) et totalisait, en fin de parcours, **plus de deux cents scénarios** de test, tout en éliminant la fragilité structurelle de l'ancienne suite : les scénarios étant purement déclaratifs, l'échec d'un test signale désormais une régression du moteur, et non un défaut du test. C'est toute la valeur du filet de sécurité qui s'en trouve restaurée : l'équipe peut faire confiance au verdict de la suite, condition première pour que les tests soient réellement consultés et maintenus.

Au regard du fil rouge (chapitre 4), cette mission actionne le levier central identifié par Feathers [2] : doter un code legacy de tests pour rendre possible toute intervention ultérieure. Le framework est, de fait, devenu une infrastructure : il a servi de socle à l'extension VS Code (section 6.2) puis au portage vers les forks legacy (section 6.3), où il a fait du filet de non-régression un préalable aux correctifs de stabilisation. Représentant à terme de l'ordre de 30 000 lignes de code, il constitue la réalisation la plus structurante de mon passage chez Free Mobile.

6.2 Outillage du framework : une extension VS Code (LSP et débogueur)

6.2.1 Contexte et dette technique initiale

Une fois le framework de test livré (section 6.1), un nouveau frein est apparu : la rédaction des tests elle-même. Les scénarios sont décrits en YAML, support sur lequel l'écriture ne bénéficie d'aucune assistance contextuelle. Là où des langages comme Python, Lua ou C++ disposent d'un serveur de langage (Language Server Protocol (LSP)) offrant autocomplétion et analyse syntaxique au fil de la frappe, le YAML n'est outillé que pour la validation de schémas. Il ne propose pas d'assistance qui tienne compte du contexte, c'est-à-dire capable de savoir si l'on est en train de saisir une instruction ou l'un de ses arguments.

Or le format de test n'est pas un simple document de configuration : c'est en réalité un langage à part entière, bâti au-dessus du YAML. On y manipule des **instructions** (équivalentes à des fonctions), des **constantes** ainsi que des **variables** qui constituent les arguments passés à ces instructions. Faute d'outillage tenant compte de cette grammaire, les erreurs de syntaxe n'étaient découvertes qu'à l'exécution et aucun moyen n'existait d'observer pas à pas le déroulé d'un scénario. Cette friction freinait l'adoption du framework par l'équipe : une dette « d'outillage ».

6.2.2 Objectif de la mission

La mission a consisté à développer une extension *Visual Studio Code*, écrite en **TypeScript**, au service du framework de la section 6.1. Cette extension réunit deux facettes complémentaires :

- un **serveur de langage** (LSP) apportant autocomplétion, diagnostics en temps réel et aide à la saisie. En s'appuyant sur la grammaire du format de test (instructions, constantes et variables), il fournit une assistance *contextuelle* : il sait distinguer l'endroit où l'on attend une instruction de celui où l'on saisit un argument, et propose les complétions correspondantes ;
- un **débogueur** permettant d'exécuter un test puis, lorsqu'une instruction échoue, d'inspecter l'état du *Valo Engine* au moment précis de l'échec, ou de progresser *pas à pas* pour comprendre le comportement interne du moteur en fonction des trames envoyées par le test.

Une fonctionnalité particulièrement utile en a découlé : la possibilité de lancer le *Valo Engine* en **mode débogage** et de progresser pas à pas *à l'intérieur même de son code C++*, tout en rejouant un scénario d'appel (*call flow*) étape par étape. Le diagnostic ne se limite ainsi plus au test, mais descend jusqu'au code natif du logiciel sous test.

6.2.3 Conception et choix techniques

Le choix du LSP répond à un principe de **découplage** entre l'éditeur et le langage : la logique d'analyse et de complétion est portée par un serveur indépendant, réutilisable au-delà de VS Code par tout éditeur compatible. Le serveur exploite la grammaire du format de test, celle-là même qu'analyse le parseur du framework (section 6.1), pour produire des complétions et des diagnostics cohérents avec ce que le framework acceptera réellement à l'exécution.

Le débogueur, lui, repose sur le **Debug Adapter Protocol** (Debug Adapter Protocol (DAP)), protocole standardisé par VS Code pour piloter une session de débogage au moyen d'échanges *JSON-RPC*. Côté framework, son intégration s'appuie sur un module qui contrôle le flot d'exécution du test (mise en pause, reprise, progression pas à pas), tout en remontant à l'éditeur les données nécessaires au diagnostic.

Les seules données internes au framework de test ne suffisent toutefois pas à diagnostiquer l'ensemble des problèmes : il faut aussi connaître l'état interne du moteur. C'est le **framework** qui centralise ces informations : il interroge le *Valo Engine* au travers d'une **API privée** et de plusieurs API publiques du moteur, dont un **service** dédié, développé par David, qui expose l'état de ses variables internes. L'extension, elle, ne dialogue qu'avec le framework, sur l'unique canal du DAP : c'est ce dernier qui lui restitue aussi bien l'état du test que celui du moteur, valeurs ensuite affichées directement dans l'Integrated

Development Environment (IDE).

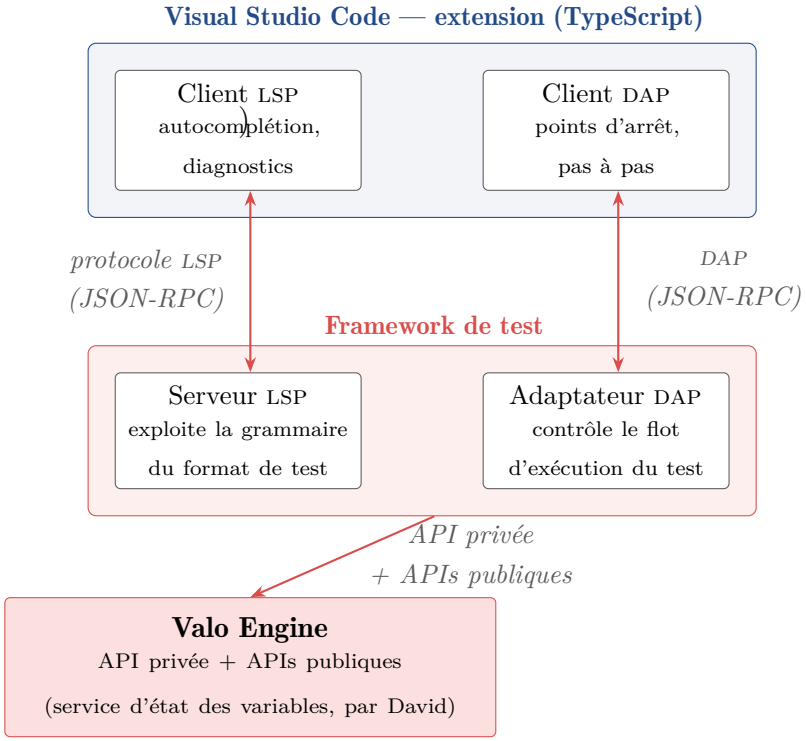


Figure 6.2 – Architecture de l'extension. VS Code dialogue avec le framework de test sur deux canaux *JSON-RPC* : le LSP pour l'assistance à l'édition, le DAP pour le débogage. En position de pivot, le framework interroge le *Valo Engine* via ses API et restitue à l'éditeur aussi bien l'état du test que celui du moteur.

6.2.4 Réalisation

Côté LSP, le serveur s'abonne aux requêtes de complétion émises par l'éditeur. À partir de la position du curseur, il détermine le contexte grammatical courant et renvoie la liste des propositions adaptées : instructions disponibles ou variables déjà déclarées.

Côté débogueur, l'adaptateur DAP traduit les ordres de l'éditeur (poser un point d'arrêt, exécuter l'instruction suivante) en commandes pilotant le flot d'exécution du test, puis renvoie l'état courant : instruction en cours, ainsi que l'état du *Valo Engine* que le framework a préalablement recueilli auprès du moteur via ses API.

6.2.5 Difficultés rencontrées

La première difficulté a tenu à la **documentation de l'API d'extension de VS Code**. Si des exemples d'extensions existent, l'API elle-même reste inégalement documentée, et aucun standard ne s'impose réellement quant à la manière de structurer le code. Les exemples disponibles sont fréquemment **monolithiques**, ce qui offre peu de modèles d'architecture réutilisables et a imposé un travail d'expérimentation pour aboutir à une organisation

maintenable.

La seconde difficulté a porté sur l'**intégration du DAP** au framework de test. Il a fallu adapter le protocole, fondé sur des échanges *JSON-RPC*, à un module capable de contrôler le flot d'exécution d'un test tout en remontant les données utiles au diagnostic. La restitution de l'état interne du moteur a par ailleurs supposé, côté framework, de réunir au sein d'une même session de débogage deux sources d'information (le flot d'exécution du test et l'état du moteur interrogé via ses API), l'ensemble étant ensuite relayé à l'extension sur le seul canal du DAP.

6.2.6 Résultats et bénéfice business

L'extension a déplacé la détection des erreurs **en amont de l'exécution** : les fautes de syntaxe et les instructions inconnues sont signalées au fil de la frappe, et l'autocomplétion contextuelle guide le rédacteur dans la grammaire du format de test, là où il fallait auparavant lancer le scénario pour découvrir la moindre coquille. L'écriture d'un test cesse ainsi d'être réservée à qui connaît le framework par cœur.

Le débogueur transforme, lui, le diagnostic des échecs : au lieu de déduire la cause d'une régression à partir du seul compte rendu final, on rejoue le scénario pas à pas en observant l'état du *Valo Engine* à chaque instruction, jusqu'à descendre, si besoin, dans son code C++. Ce confort n'est pas accessoire : un outil de qualité n'a d'effet sur la dette que s'il est *adopté*, et c'est précisément cette barrière d'entrée que l'extension abaisse (chapitre 4). Elle convertit le framework d'un outil d'expert en un outil d'équipe.

6.3 Portage du framework vers les forks legacy du Valo Engine

6.3.1 Contexte et dette technique initiale

Le framework de test (section 6.1) avait été conçu pour le *Valo Engine* du **réseau prépayé** de Free Mobile, un réseau alors nouveau. Une fois ce dernier mis en production et jugé **stable**, la stratégie de gestion de la dette technique a été révisée.

La stratégie initiale faisait du prépayé le socle d'une **base de code commune et moderne**, destinée à être réutilisée par les moteurs de valorisation des différents réseaux : on aurait ainsi migré progressivement chaque réseau vers cette version moderne. Elle a cédé la place à une approche de **quick fix** (que l'on pourrait presque qualifier de *quick converting*), visant à **stabiliser au maximum** les réseaux existants sans en moderniser le code ; la modernisation n'interviendrait, le cas échéant, qu'une fois cette stabilisation acquise.

Or ces réseaux reposent sur autant de fork *legacy* du moteur, chacun doté de ses spécificités métier. Plusieurs inconnues pesaient sur les exécutables réellement en production : certains binaires n'avaient pas été mis à jour depuis longtemps, et les sources se répartissaient entre le dépôt GitLab et diverses machines. Toucher à ce code sans filet exposait au risque financier rappelé en introduction de ce chapitre : la dette tenait à l'absence de tout moyen de garantir qu'un correctif n'introduise pas, par inadvertance, un **changement de logique métier** sur l'un de ces forks.

6.3.2 Objectif de la mission

La mission a consisté à **porter le framework de test** vers l'ensemble des forks legacy du *Valo Engine*, afin de doter chaque réseau d'un filet de non-régression *avant* d'y appliquer le moindre correctif. Le périmètre couvrait six variantes : la France **B2C** et **B2B**, les **Caraïbes**, l'Italie **B2C** et **B2B**, ainsi que le **proxy** italien.

L'objectif premier n'était pas de couvrir fonctionnellement chaque réseau dans les moindres détails, mais de disposer d'un **contrôle sur le code métier** de chaque fork : pouvoir y appliquer des correctifs avec l'assurance que la logique de valorisation demeurait inchangée.

6.3.3 Conception : factorisation et paramétrage

La principale difficulté de conception tenait à l'hétérogénéité des réseaux, qui se répartissent en deux modèles de facturation. Les réseaux en online charging fonctionnent comme le prépayé : les trames d'appel, de SMS et de data sont échangées en temps réel via un protocole TCP interne. Les réseaux en offline charging, eux, ne dialoguent pas en direct avec le moteur : celui-ci **consomme des CDR**, c'est-à-dire des fichiers décrivant a posteriori la consommation. Un premier chantier a donc consisté à doter le framework de la capacité à **produire et lire des CDR** pour simuler les équipements en *offline charging*, là où il ne savait jusque-là qu'émettre des trames TCP.

Pour absorber cette diversité, David avait amorcé une refonte du framework articulée autour de deux notions complémentaires :

- un **core**, socle commun à toutes les versions supportées. Il porte la mécanique invariante : la boucle d'exécution et le *parsing* des instructions du langage de test ;
- des **specs**, une par fork, qui spécifient *quelles* instructions sont disponibles et *comment* elles s'exécutent, en fournissant leur **implémentation runtime** propre au réseau visé.

Cette séparation *core* / *spec* permet de mutualiser tout ce qui ne dépend pas du réseau et de n'isoler, fork par fork, que les différences de comportement métier, ce qui prépare du même coup la modernisation ultérieure.

6.3.4 Réalisation

Le portage s'est déroulé en **binôme**. Ayant déposé ma démission, je m'inscrivais dans une démarche de **transmission de connaissances** avec Jérémy, recruté en renfort de l'équipe. Travailler à deux lui a permis de s'approprier un framework devenu conséquent (de l'ordre de 30 000 lignes, près de 40 000 en comptant le code généré pour la communication HTTP), tout en menant le portage à bien.

Concrètement, chaque fork a été décrit par sa *spec*, qui enregistre auprès du *core* l'implémentation des instructions propres au réseau ; le *core* reste, lui, identique d'un fork à l'autre.

6.3.5 Difficultés rencontrées

Deux obstacles ont dominé la réalisation. Le premier a été le **décodage du format binaire des CDR**. Ces enregistrements sont certes normalisés en amont par une brique dédiée, le *CDR hub* ; il restait néanmoins à **décoder leur contenu** : la disposition exacte des données, la valeur de chaque champ et, plus encore, ce que ces champs représentent d'un point de vue fonctionnel.

Le second a tenu au **pilotage des engines legacy**. Contrairement au moteur du prépayé, beaucoup plus moderne et doté d'un service de testing exposant une **API HTTP**, les forks legacy n'avaient **jamais été pensés pour accueillir des tests**. Il a donc fallu trouver le moyen de les **démarrer**, de les **arrêter** et de **dialoguer** avec eux, sans le point d'entrée dédié dont bénéficiait le prépayé.

6.3.6 Expérimentation : l'IA agentique au service de la transmission

La révision de la stratégie de gestion de la dette technique s'est accompagnée d'une inflexion plus large dans la manière dont Free Mobile entendait voir ses **logiciels internes développés**. L'entreprise a notamment revu sa **politique vis-à-vis des IA externes** et déployé des **licences d'IA agentique**, au premier rang desquelles **Cursor**, à l'ensemble de ses développeurs. Ce portage, mené en binôme et placé sous le signe de la **transmission de connaissances**, m'a offert un terrain d'**expérimentation** idéal pour éprouver les capacités de production réelles de ces outils. Les observations qui suivent rejoignent assez fidèlement les conclusions d'une étude d'IBM sur les impacts de l'usage des Large Language Model (grand modèle de langage) (LLM) pour produire du code [12].

Le framework reposait sur un socle *core* / *spec* (section précédente) pensé, après refonte, pour **supporter son extensibilité**. Cette qualité d'architecture s'est révélée déterminante : tant que l'on demeurait dans un **cadre déjà balisé**, c'est-à-dire décrire un nouveau fork à l'aide d'instructions *standards* sans sortir vers des fonctionnalités encore **non développées**,

il était relativement simple de **guider le LLM** vers le résultat attendu. Sur ce périmètre, le gain de **productivité** fut considérable.

Deux facteurs sont toutefois apparus comme **conditionnant** ce gain.

Le premier tient à la **maîtrise préalable de l'architecture**. Ayant réalisé l'essentiel du framework (quoique d'autres développeurs y aient également contribué), j'en avais une **vision fine**, ce qui rendait **aisée la rédaction de *prompts* adéquats** pour obtenir précisément ce que je visais. L'IA **amplifiait** ici une compréhension déjà constituée plutôt qu'elle ne s'y substituait.

Le second facteur est apparu *a contrario*, lors des recherches sur le décodage des CDR. Ce travail a en partie été conduit par Jérémy **en mon absence**, durant une période de cours, à l'aide de la solution **agentique d'OpenAI**. Les résultats se sont révélés **décevants**, et même **plus complexes** que la solution finalement retenue. Ce relatif échec s'explique avant tout par un **manque de connaissance**, à la fois du **code métier** sous-jacent et de l'**architecture du framework**, qui empêchait de formuler les instructions justes et de juger sainement les propositions du modèle.

Un troisième facteur, plus insidieux, a pesé sur l'expérimentation : l'IA a considérablement **accru la difficulté des revues de code**. Le framework de test avait été développé de façon relativement **isolée**, surtout depuis le départ de David Desmarest : peu de personnes dans l'équipe en conservaient une **connaissance fine**. Or un LLM produit de **grands volumes de code d'un seul tenant**, ce qui rendait les modifications **ardues à relire**, au point de soulever une question vertigineuse : ne faudrait-il pas **confier la revue à l'IA** elle-même ? Une telle délégation se ferait pourtant au **détriment de la transmission des connaissances** et de l'**harmonisation** du style d'écriture du code, deux garde-fous pourtant essentiels contre l'accumulation d'une nouvelle dette.

J'en retire un constat qui fait directement écho au fil rouge de ce mémoire : le LLM agit comme un **levier**, non comme un substitut. Sa productivité est **proportionnée à la maîtrise** que possède celui qui le pilote, de l'architecture comme du domaine métier, et à la **qualité du cadre** dans lequel il opère. Une base bien architecturée et extensible, c'est-à-dire **peu endettée**, démultiplie son apport ; à l'inverse, sur un terrain *legacy* ou opaque, mal maîtrisé, il tend à produire un code **plus complexe** et moins fiable. L'outil ne dispense donc pas de la **connaissance** : il la **récompense**.

6.3.7 Résultats et bénéfice business

Le portage a étendu le filet de non-régression à l'ensemble des réseaux du *Valo Engine* : **sept réseaux** au total, soit les six forks legacy (France B2C et B2B, Caraïbes, Italie B2C et B2B, proxy italien) et le réseau prépayé. Les correctifs de stabilisation pouvaient désormais

être appliqués avec l'assurance qu'aucun **changement de logique métier** ne s'y glissait à l'insu de l'équipe. Sur un composant dont une défaillance se chiffre en millions d'euros par semaine, ce contrôle constitue avant tout une **maîtrise du risque financier**.

Quantifier le nombre de régressions ainsi « évitées » serait toutefois trompeur : le framework s'exécutant en CI **avant tout *merge***, un défaut introduit par un correctif est intercepté avant même d'être intégré à la branche principale. Il ne devient donc jamais une régression au sens strict, puisqu'il n'atteint ni l'intégration ni la production. C'est précisément l'effet recherché : le bénéfice ne se mesure pas en incidents corrigés, mais en incidents qui n'ont jamais eu lieu.

Au-delà de ce gain, l'architecture *core* / *spec* factorise la mécanique commune et n'isole que les spécificités de chaque réseau : elle réduit la duplication entre forks et **prépare la modernisation** envisagée à plus long terme (chapitre 4).

Chapitre 7

Développer du code à visée de production dans un existant endetté

Ce dernier chapitre, consacré à une mission menée chez Beemo, change de posture : il ne s'agit plus d'outiller ou de tester l'existant, mais d'**écrire du code neuf destiné à la production** au cœur d'un firmware massivement endetté : fichiers de plusieurs dizaines de milliers de lignes, couplage généralisé, absence de tests (chapitre 4).

L'enjeu est de livrer la fonctionnalité **sans accroître la dette**, et la difficulté est double. Technique, d'abord : chaque point de contact avec le legacy importe son opacité et son couplage ; du code neuf écrit « dans le style » de l'existant en hérite mécaniquement, et l'absence de filet de test rend toute intégration risquée. Méthodologique, ensuite : l'existant exerce une pression de conformité, car le chemin de moindre effort consiste à imiter ce qui est déjà là, donc à reproduire les pratiques qui ont créé la dette. S'en prémunir suppose une discipline d'**isolation** : écrire le neuf de manière testable et découplée, et confiner le dialogue avec le legacy dans une couche d'adaptation explicite, plutôt que de laisser les deux mondes s'entremêler.

La section 7.1 illustre cette démarche avec le développement d'une fonctionnalité d'immuabilité des sauvegardes, dont l'architecture trace une voie de modernisation progressive du firmware.

7.1 Développement d'une fonctionnalité en code de production

7.1.1 Contexte et besoin

Le métier de Beemo n'est pas de proposer un **service de cloud** à la manière des grands fournisseurs que sont Amazon, Google ou Microsoft. Il s'agit de fournir aux **professionnels**, c'est-à-dire aux hébergeurs et aux entreprises pour qui la donnée constitue un actif critique, un moyen de **sécuriser leurs données** et de garantir leur **restauration** en cas d'incident, qu'il soit d'origine **physique** ou **malveillante**. Le produit central, la **Beebox**, est une appliance de **sauvegarde** qui peut être **externalisée** hors site afin de survivre à la destruction du système d'origine.

L'un des scénarios de perte de données les plus redoutés est le **ransomware**, qui chiffre les données d'un système pour en exiger une rançon. La Beebox répond directement à cette menace en conservant des **jeux de sauvegarde** hors d'atteinte du système compromis. Mais une question demeure : **que se passe-t-il si la Beebox elle-même est compromise** ? Un attaquant parvenu jusqu'à l'appliance pourrait alors tenter d'**altérer les jeux de sauvegarde** ou de **saboter leur restauration**, anéantissant ainsi la dernière ligne de défense du client.

C'est pour répondre à ce scénario que Beemo développe une nouvelle fonctionnalité, l'**immuabilité**. Son principe : rendre un jeu de sauvegarde **insupprimable** et **inaltérable** tant qu'un **délai de rétention** n'est pas écoulé, y compris pour un utilisateur disposant de droits sur la box, et donc pour un attaquant qui en aurait pris le contrôle. J'ai participé à son développement aux côtés de **deux autres développeurs**, en me concentrant sur la partie **embarquée sur la box**, écrite en **C++**.

7.1.2 Objectif de la mission

L'objectif n'était pas seulement de **livrer la fonctionnalité** : comme l'a posé l'introduction de ce chapitre, il fallait le faire **sans aggraver la dette** d'un firmware qui en cumule la plupart des symptômes :

- un **abus de directives de préprocesseur** (`#ifdef`, `#ifndef`) pour gérer les multiples cas de **compilation** de la box, rendant le code difficile à lire et à raisonner ;
- des **fonctions extrêmement longues** et une **maîtrise approximative du C++**, donnant lieu à des constructions trompeuses, par exemple des contrôles du type `this == nullptr`, un cas qui **ne peut jamais se produire** et trahit une incompréhension du langage ;
- une **absence totale de tests** (ni test de non-régression, ni tests unitaires), qui prive de tout filet de sécurité lors d'une modification ;

— un fichier `main.h` qui **inclut l'ensemble des fichiers** du projet, créant un **couplage massif** : réutiliser une fonctionnalité existante depuis du code neuf devient une opération complexe et risquée.

La contrainte centrale était donc d'écrire du code **moderne, testable et isolé**, tout en **dialoguant** avec un existant qui ne s'y prête pas.

7.1.3 Conception et intégration

La réponse à cette tension repose sur une **couche intermédiaire** d'**adaptation**, conçue par David, l'un des développeurs de l'équipe. L'idée est d'**encapsuler** une fonctionnalité, qu'elle soit **legacy** ou **nouvelle**, derrière une **classe** pouvant être appelée **indifféremment depuis l'ancien code ou depuis le code neuf**. Les appels vers l'existant sont **masqués** par des **pointeurs** transmis à cette classe, ce qui **découple** le nouveau code du couplage imposé par `main.h`.

Ce mécanisme d'*enrobage* (*wrapping*) joue le rôle de **frontière** : d'un côté, le firmware historique et ses contraintes ; de l'autre, un espace où l'on peut écrire du **C++ réellement moderne** pour réaliser la fonctionnalité. La fonctionnalité d'immuabilité a ainsi pu être bâtie autour de deux briques propres :

— un **automate à états** (*state machine*) qui pilote le cycle de vie d'un jeu de sauvegarde immuable : création, période de rétention, mise en pause du **planificateur de sauvegarde**, expiration ;

— un **service de chiffrement** qui garantit l'**intégrité** des données. La base est **lue et déchiffrée à l'aide d'une clé** : si elle a été **altérée**, le déchiffrement échoue, **le code le détecte** et la box se **bloque**, ce qui empêche toute manipulation des jeux de données réels par un attaquant parvenu jusqu'à l'appliance.

L'immuabilité vient ainsi s'ajouter aux mécanismes de sécurisation existants : elle **empêche la suppression** d'un jeu de sauvegarde considéré comme immuable tant que le **délai de rétention** n'a pas été respecté.

7.1.4 Réalisation

La couche d'adaptation est le point d'entrée de toute la fonctionnalité : une classe moderne expose le service d'immuabilité au reste du système, tout en dissimulant les appels au firmware historique derrière un pointeur opaque transmis à la construction. Ce procédé est l'idiome **PIMPL** (*Pointer to Implementation*) : l'en-tête public ne déclare qu'une classe d'implémentation *anticipée* et un `std::unique_ptr` vers celle-ci, sans rien révéler ni inclure de l'existant (listing 7.1).

```cpp
// immutability.h - exposed to new code, does NOT include main.h
#include <memory>

class Immutability {
public:
    Immutability();
    ~Immutability();

    void lock(BackupSet& set, Duration retention);

private:
    class Impl;                    // forward declaration only
    std::unique_ptr<Impl> pimpl;   // opaque pointer to the
        implementation
};
```

Listing 7.1 – En-tête de la couche d'adaptation : aucune dépendance vers `main.h`.

Tout le couplage avec le legacy est alors **confiné à un seul fichier source** (listing 7.2) : c'est la seule unité de traduction qui inclut `main.h`. Le code client, lui, ne compile plus que contre une interface propre, et une modification interne du firmware n'entraîne pas sa recompilation.

```cpp
// immutability.cpp - the only translation unit aware of the legacy
    firmware
#include "immutability.h"
#include "main.h"  // legacy coupling confined here

class Immutability::Impl {
public:
    void lock(BackupSet& set, Duration retention) {
        legacy_pause_scheduler();  // call into the historic firmware
        // ... modern logic: state machine, encryption
    }
};

Immutability::Immutability() : pimpl(std::make_unique<Impl>()) {}
Immutability::~Immutability() = default;  // defined where Impl is
    complete

void Immutability::lock(BackupSet& set, Duration retention) {
    pimpl->lock(set, retention);
}
```

Listing 7.2 – Source de la couche d'adaptation : le legacy est confiné à cette unité de traduction.

Le destructeur est volontairement défini dans le fichier source, là où le type `Impl` est *complet* : c'est la condition pour que `std::unique_ptr` sache détruire un type qui n'est qu'*anticipé* dans l'en-tête. Ce détail résume l'esprit de la mission : écrire un C++ moderne et correct *au contact* d'un existant endetté, sans en propager les dépendances.

7.1.5 Difficultés rencontrées

La principale difficulté n'a pas été la **conception de la fonctionnalité** en elle-même, mais sa **cohabitation avec l'existant**. Comprendre le firmware historique, afin de **réutiliser ses fonctionnalités** depuis le code neuf, s'est avéré coûteux, en raison du **couplage par `main.h`**, des **fonctions interminables** et des **cas de compilation** multipliés par les directives de préprocesseur.

L'**absence de tests** a constitué un risque permanent : sans test de non-régression, la moindre modification dans une zone partagée pouvait **casser silencieusement** une fonctionnalité existante. La couche d'adaptation a ici joué un rôle de **garde-fou**, en **circonscrivant** les points de contact avec le legacy et en rendant le code neuf **testable de manière isolée**, là où le reste du firmware ne l'est pas.

7.1.6 Résultats et bénéfice business

Cette tâche illustre une facette essentielle de la lutte contre la dette technique : il ne s'agit pas toujours de **résorber** l'existant, mais parfois de savoir **construire proprement à côté** sans l'aggraver. Grâce à la couche d'adaptation, la fonctionnalité d'immuabilité a pu être développée en **C++ moderne**, **isolée** et **testable**, alors même qu'elle s'insère dans un firmware fortement dégradé et dépourvu de filet de sécurité.

Sur le plan **business**, l'immuabilité renforce directement la **proposition de valeur** de Beemo : elle protège les jeux de sauvegarde non seulement contre la compromission du système client, mais aussi contre la **compromission de la Beebox elle-même**. En garantissant qu'un jeu de sauvegarde immuable ne peut être ni supprimé ni altéré avant l'expiration de son **délai de rétention**, et en **bloquant** la box au moindre signe d'altération détecté par le chiffrement, la fonctionnalité ferme l'un des derniers angles d'attaque possibles et consolide la promesse fondamentale du produit : **la donnée sauvegardée n'est jamais perdue**.

Chapitre 8

Conclusion

8.1 Bilan fonctionnel

Au moment de la rédaction de ce mémoire, l'alternance n'est pas achevée : les missions menées chez Beemo Technologie sont encore en cours. Le bilan qui suit photographie donc l'état des travaux à la date de la soutenance, en reprenant les trois volets thématiques qui structurent ce mémoire.

Sur le volet de l'**outillage et de la standardisation de l'environnement de développement** (chapitre 5), l'outil de duplication Grafana (section 5.1) est livré et exploité pour synchroniser les instances de supervision. L'étude sur le coût de compilation (section 5.2) a rendu le *build* du *Valo Engine* soutenable sur les machines de développement, tout en documentant la dette de fond que seuls les modules C++20 résorberont à terme. La rétro-ingénierie du DSDK (section 5.3) a atteint son objectif principal, un *build* local, reproductible et versionné, mais reste ouverte sur la validation du package affecté par l'un des correctifs.

Sur le volet de la **testabilité du logiciel legacy** (chapitre 6), les trois missions ont été menées à leur terme et livrées. Le framework de test (section 6.1) a remplacé la suite Lua sur l'ensemble de son périmètre et constitue désormais le filet de non-régression du *Valo Engine* ; l'extension VS Code (section 6.2) l'accompagne et en a ouvert l'usage à toute l'équipe ; le portage vers les forks legacy (section 6.3) a couvert les sept réseaux visés (les six forks legacy et le réseau prépayé) et a été l'occasion d'une transmission de connaissances en binôme avant mon départ.

Sur le volet du **code à visée de production dans un existant endetté** (chapitre 7), la fonctionnalité d'immuabilité (section 7.1) est en cours de développement : ses briques centrales (automate à états, service de chiffrement, couche d'adaptation au firmware) sont en place, le code neuf étant écrit de manière testable et isolée, à rebours des pratiques du

firmware qui l'accueille.

8.2 Retour sur le fil rouge : la dette technique

Relues à travers la grille du chapitre 4, ces sept missions dessinent une conclusion d'ensemble : la dette technique ne se rembourse pas par un grand soir, mais par une accumulation d'actions ciblées, chacune actionnant un levier précis : outiller, tester, fiabiliser la chaîne de production, isoler le neuf de l'ancien. Dans les deux entreprises, la stratégie gagnante a été *incrémentale* : préserver la capacité à livrer tout en réduisant, intervention après intervention, le risque et le coût des suivantes.

J'en retiens aussi deux enseignements moins attendus. Le premier est que la forme de dette la plus dangereuse n'est pas dans le code : c'est la dette de connaissance, celle qui part avec les personnes, comme le framework de test après le départ de David ou la procédure de compilation de l'agent ESX que je suis venu reprendre. Sa parade est la capitalisation sous forme exécutable : tests, patchs versionnés, outillage. Le second est la dimension humaine du remboursement : un travail de fond sur du legacy est ingrat, et la motivation des équipes en est un facteur limitant au même titre que la technique. Les outils d'IA agentique, expérimentés en fin de parcours (section 6.3), ne changent pas cette équation : ils récompensent la maîtrise de l'architecture et du métier, mais ne s'y substituent pas.

8.3 Apports de l'alternance

Sur le plan technique, ces deux années m'ont fait pratiquer un spectre inhabituellement large pour un alternant : le C++ d'un moteur critique et d'un firmware embarqué, le Python d'un framework de test de 30 000 lignes conçu et architecturé de bout en bout, le TypeScript d'une extension VS Code, les protocoles LSP et DAP, la rétro-ingénierie d'un SDK propriétaire, Docker, et les arcanes d'une chaîne de compilation C++. Plus que les technologies elles-mêmes, j'en retire une compétence transversale : savoir aborder un système existant, opaque et non documenté, et en reprendre le contrôle méthodiquement.

Sur le plan organisationnel et humain, travailler dans deux entreprises, un grand opérateur et une PME d'une trentaine de personnes, m'a donné deux points de comparaison sur la méthode agile, la gestion d'une équipe face au legacy, et la place qu'un développeur peut y occuper : transverse et hors sprint chez Free Mobile, pleinement intégré aux livraisons chez Beemo. Le portage en binôme m'a par ailleurs placé en situation de *transmettre*, un exercice qui force à expliciter ce que l'on croyait évident. Enfin, sur le plan personnel, la décision de quitter Free Mobile pour suivre une nouvelle opportunité m'a appris à arbitrer un choix de carrière, et la confiance accordée sur des missions structurantes a confirmé mon goût pour l'ingénierie au service de la qualité logicielle.

8.4 Perspectives

Plusieurs chantiers prolongent naturellement ces travaux. Chez Free Mobile, l'architecture *core/spec* du framework prépare la modernisation des forks legacy vers un socle commun, le filet de non-régression rendant désormais cette migration envisageable ; la résorption de la dette de compilation passera, elle, par l'adoption des modules C++20 lorsque les chaînes de compilation l'autoriseront. Chez Beemo, la suite est double : fiabiliser la validation des packages auprès de Broadcom, puis l'automatiser pour achever l'harmonisation de la chaîne de production de l'agent ESX, et mener à son terme la fonctionnalité d'immuabilité. À plus long terme, la couche d'adaptation qui l'héberge ouvre une voie de modernisation progressive du firmware : écrire chaque évolution en C++ moderne et testable, et « gommer » le legacy fonctionnalité par fonctionnalité.

Glossaire des acronymes

API Application Programming Interface.

CD Continuous Delivery / Deployment.
CDR Call Detail Record (enregistrement détaillé d'appel).
CI Continuous Integration (intégration continue).

DAP Debug Adapter Protocol.
DSDK Daemon SDK (SDK propriétaire VMware/Broadcom pour les daemons ESXi).

HTTP HyperText Transfer Protocol.

IDE Integrated Development Environment.

JSON JavaScript Object Notation.

LLM Large Language Model (grand modèle de langage).
LSP Language Server Protocol.

PoC Proof of Concept.

RPC Remote Procedure Call (appel de procédure distante).

SAX Simple API for XML (analyse événementielle, au fil du flux).
SDK Software Development Kit.

TCP Transmission Control Protocol.
TUI Text User Interface (interface en mode texte).

YAML YAML Ain't Markup Language (format de sérialisation lisible).

Glossaire

chaîne de compilation (toolchain) Ensemble des outils (compilateur, éditeur de liens, bibliothèques) permettant de transformer le code source en binaire exécutable pour une cible donnée.

dette technique Coût implicite des retouches futures induit par le choix d'une solution rapide plutôt que d'une approche meilleure mais plus longue à mettre en œuvre. Se rembourse par le refactoring ; génère des « intérêts » sous forme de ralentissement du développement.

fork Divergence d'une base de code en plusieurs variantes maintenues en parallèle, partageant une origine commune.

IA agentique Assistant fondé sur un LLM capable d'agir de façon autonome sur une base de code (lire les fichiers, exécuter des commandes, rédiger et modifier du code en plusieurs étapes) à partir d'instructions en langage naturel (*prompts*). Cursor en est un exemple intégré à l'éditeur.

offline charging Modèle de facturation a posteriori : la consommation est décrite dans des fichiers (CDR) que le moteur de valorisation consomme après coup, sans dialogue en temps réel avec les équipements.

online charging Modèle de facturation en temps réel : les équipements interrogent le moteur de valorisation pendant l'appel (ou l'envoi de SMS/data) via un protocole d'échange direct, ici un protocole TCP interne.

rétro-ingénierie Analyse d'un système ou binaire existant pour en comprendre la structure et le comportement, en l'absence de documentation ou de sources complètes.

test de non-régression Test vérifiant qu'une modification du code n'a pas réintroduit de défaut ni cassé une fonctionnalité existante.

Bibliographie

[1] Ward Cunningham. The WyCash portfolio management system. OOPSLA '92 Experience Report, 1992. Origine de la métaphore de la « dette technique ».

[2] Michael Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004.

[3] Martin Fowler. Technical debt quadrant. martinfowler.com (bliki), 2009. `https://martinfowler.com/bliki/TechnicalDebtQuadrant.html`.

[4] Martin Fowler. *Refactoring : Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018.

[5] GNU Project. Gcc 15 release series — changes, new features, and fixes. gcc.gnu.org, 2025. `https://gcc.gnu.org/gcc-15/changes.html`, consulté le 13 juin 2026.

[6] Jez Humble and David Farley. *Continuous Delivery*. Addison-Wesley, 2010.

[7] Nada Kanita. Modèle explicatif du phénomène de la dette technique dans le contexte agile : revue de littérature qui mobilise la méthode bibgt. ResearchGate, 2022. `https://www.researchgate.net/publication/361219702`.

[8] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. Technical debt : From metaphor to theory and practice. *IEEE Software*, 29(6) :18–21, 2012.

[9] Jesper Olsson, Erik Risfelt, Terese Besker, Antonio Martini, and Richard Torkar. Measuring affective states from technical debt : A psychoempirical software engineering experiment. *Empirical Software Engineering*, 26(5) :105, 2021. DOI : 10.1007/s10664-021-09998-w.

[10] Luka Pavlič, Tilen Hliš, Marjan Heričko, and Tina Beranič. The gap between the admitted and the measured technical debt : An empirical study. *Applied Sciences*, 12(15) :7482, 2022. DOI : 10.3390/app12157482.

[11] Pytest Development Team. pytest documentation. Documentation officielle du framework de test pytest. `https://docs.pytest.org/`, consulté le 12 juin 2026.

[12] Cole Stryker. Réduire la dette technique en 2026. IBM Think Insights, 2026. `https://www.ibm.com/fr-fr/think/insights/reduce-technical-debt`, consulté le 10 juin 2026.

[13] Damian A. Tamburri, Philippe Kruchten, Patricia Lago, and Hans van Vliet. Social debt in software engineering : Insights from industry. *Journal of Internet Services and Applications*, 6(1) :10, 2015. DOI : 10.1186/s13174-015-0024-6.

[14] Textualize. Textual documentation. Documentation officielle du framework Python Textual. `https://textual.textualize.io/`, consulté le 12 juin 2026.

Annexe A

Annexes

A.1 Chronologie des missions

Le diagramme ci-dessous restitue, au mois près, la chronologie réelle des missions sur les deux années d'alternance, le corps du mémoire les présentant quant à lui regroupées par nature. Débutée le 8 juillet 2024 chez Free Mobile, l'alternance s'achève fin août 2026 ; elle s'est poursuivie chez Beemo Technologie à partir d'avril 2026, où les deux missions ont été menées de front, avec les interruptions et reprises inhérentes à ce type de chantier ; elles sont encore en cours au moment de la rédaction. S'y ajoute, de janvier à mars 2026, une parenthèse consacrée à des correctifs sur le code de production de Free Mobile, menée en parallèle du portage.

La dernière ligne du diagramme rappelle le rythme de l'alternance : les périodes de cours et d'examens (M1 en 2024–2025, M2 en 2025–2026) s'arrêtent en janvier 2026, l'entreprise occupant le reste du temps, dont la totalité des étés ; à partir de mi-décembre 2025, la présence en entreprise devient continue.

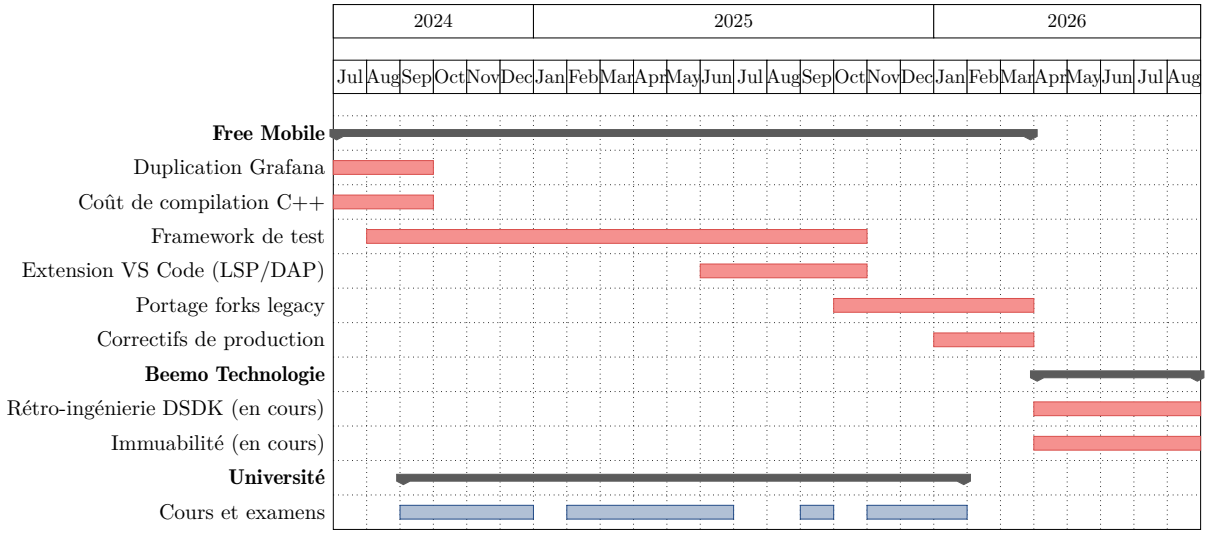


Figure A.1 – Chronologie des missions de l'alternance.